

Contents

1.1	what is the difference between data and information.	2
1.2	How to transfer raw data into information in Database Management System?	3
1.3	What is database and database management system	5
1.4	What is the main role and advantage of DBMS?	7
1.5	Database application.	9
1.6	How DBMS manages the interaction between the end user and the database	11
1.7	types of database in different things.	13
1.8	purpose of database system.	15
1.9	why database design is important.....	16
1.10	Environment of database management system.....	17
1.11	Function of database.	19
1.12	Abstraction of database management system.....	20
2.0	What is metadata?	22
2.1	Historical roots of file and file system in DBMS.....	24
2.2	What is data?	27
2.3	What is field?	28
2.4	What is record?.....	29
2.5	What is file?	31
2.6	Make a customer file with all proper content.	32
3.1	what is entity set?	33
3.2	What is relationship set?	34
3.3	What is recursive relation?	35
3.4	What is er diagram? The content of er diagram.	37
3.5	Describe about mapping cardinalities? In details.	39
3.6	What is super key, primary key and candidate key and foreign key?.....	41
3.7	What is Binary relation? With example.	44
3.8	What is ternary relation? With example.	45
3.9	Describe the process of replacing ternary relation.....	47
4.1	What is DDL? possible all example based on DBMS MS SQL.	49
4.2	What is DML? possible all example based on DBMS MS SQL.	51
4.3	Create a database for an university and differentiate the DDL and DML using MS SQL.	54
5.0	Database management by the programing language.	57
5.1	how to create a database from story problem set including creating ER diagram.....	57
5.2	How to identify the entity?.....	61
5.3	How to identify the attribute?	61

5.4 How to identify the relationship?	61
5.5 How to identify the cardinality?	61
5.6 How to take the primary key?	63
5.7 How to make relation and put in ER diagram?	64
5.8 In a relationship how many tables we need including main data tables and can we reduce it.....	65
5.9 Difference between truncate and drop.	67
5.10 What is subquery and its practice example.....	68
5.11 What is arithmetic function?	70
5.12 What is union and other join operation?.....	72
5.13 Join vs union operation (for 3*3 tow table).	75
5.14 Join vs set operation.	78
5.15 What is.....	80

1.1 what is the difference between data and information.

✅ Answer in English

Data and **Information** are closely related but not the same.

◆ Data

- Data refers to **raw facts and figures**.
- It is **unprocessed**, unorganized, and has no meaning by itself.
- Data can be numbers, text, symbols, or observations.
- Example: 50, 60, 70, or names like Rahim, Karim

👉 Data alone does not help in decision-making.

◆ Information

- Information is **processed and organized data**.
- It has **meaning and context**, making it useful.
- Information is used for **decision-making**.
- Example: "The average marks of the class is 60"

👉 Information helps us understand something clearly.

🔄 Key Differences

Aspect	Data	Information
Meaning	Raw facts	Processed data with meaning
Nature	Unorganized	Organized
Use	Not directly useful	Useful for decision-making
Example	80, 90, 75	Average score = 81.6

📌 Simple Example

- Data: 10, 20, 30

- Information: “The total is 60 and average is 20”

✓ বাংলা উত্তর (Bangla)

Data এবং **Information** একে অপরের সাথে সম্পর্কিত হলেও এক জিনিস না।

◆ ডাটা (Data)

- ডাটা হলো **কাঁচা তথ্য বা সংখ্যা**।
- এটি **অপ্রক্রিয়াজাত (unprocessed)** এবং কোনো অর্থ বহন করে না।
- ডাটা হতে পারে সংখ্যা, লেখা, বা চিহ্ন।
- উদাহরণ: ৫০, ৬০, ৭০ বা রহিম, করিম

👉 শুধু ডাটা দিয়ে কোনো সিদ্ধান্ত নেওয়া যায় না।

◆ তথ্য (Information)

- তথ্য হলো **প্রক্রিয়াজাত (processed) ডাটা**।
- এটি **অর্থপূর্ণ এবং সাজানো থাকে**।
- তথ্য **সিদ্ধান্ত নিতে সাহায্য করে**।
- উদাহরণ: “ক্লাসের গড় নম্বর ৬০”

👉 তথ্য আমাদের পরিষ্কারভাবে বুঝতে সাহায্য করে।

📁 মূল পার্থক্য

বিষয়	ডাটা (Data)	তথ্য (Information)
অর্থ	কাঁচা তথ্য	প্রক্রিয়াজাত অর্থপূর্ণ তথ্য
অবস্থা	অসংগঠিত	সংগঠিত
ব্যবহার	সরাসরি কাজে লাগে না	সিদ্ধান্ত নিতে সাহায্য করে
উদাহরণ	৮০, ৯০, ৭৫	গড় = ৮১.৬

✦ সহজ উদাহরণ

- ডাটা: ১০, ২০, ৩০
- তথ্য: “মোট ৬০ এবং গড় ২০”

1.2 How to transfer raw data into information in Database Management System?

✓ Answer in English

In a **Database Management System (DBMS)**, raw data is transformed into meaningful **information** through several organized steps.

◆ Steps to Convert Data into Information

1. Data Collection

- Raw data is collected from different sources (users, sensors, transactions, etc.).
- Example: Student marks, names, IDs.

2. Data Storage

- The collected data is stored in a **database** using tables.
- Data is organized into **rows (records)** and **columns (fields)**.
- Example: A “Student” table with columns like ID, Name, Marks.

3. Data Processing

- DBMS processes the data using operations like:
 - **Sorting** (arranging data)
 - **Filtering** (selecting specific data)
 - **Calculating** (sum, average, etc.)
- Example: Calculating average marks.

4. Querying (Using SQL)

- Users retrieve specific data using **queries** (SQL commands).
- Example:
 - SELECT AVG(Marks) FROM Student;
 - This converts raw marks into meaningful results.

5. Data Analysis

- The processed data is analyzed to find patterns or insights.
- Example: Finding top students or pass/fail rate.

6. Information Output

- Finally, the result is presented as **reports, charts, or summaries**.
- This output is meaningful and useful → this is **Information**.

🔴 Simple Example

- Raw Data:
50, 60, 70
- After Processing:
“Average marks = 60” → **Information**

📁 Summary

👉 Data → Stored → Processed → Queried → Analyzed → Information

✅ বাংলা উত্তর (Bangla)

একটি **Database Management System (DBMS)**-এ কাঁচা ডাটা (raw data) ধাপে ধাপে প্রক্রিয়াজাত হয়ে **তথ্য (information)**-এ পরিণত হয়।

♦ ডাটা থেকে তথ্য তৈরির ধাপগুলো

১. ডাটা সংগ্রহ (Data Collection)

- বিভিন্ন উৎস থেকে কাঁচা ডাটা সংগ্রহ করা হয়।
- উদাহরণ: শিক্ষার্থীর নাম, রোল, নম্বর।

২. ডাটা সংরক্ষণ (Data Storage)

- ডাটাকে টেবিল আকারে ডাটাবেজে সংরক্ষণ করা হয়।

- সারি (row) এবং কলাম (column) আকারে সাজানো থাকে।
- উদাহরণ: Student টেবিল।

৩. ডাটা প্রক্রিয়াকরণ (Data Processing)

- ডাটার উপর বিভিন্ন কাজ করা হয়:
 - সাজানো (sorting)
 - বাছাই করা (filtering)
 - হিসাব করা (average, sum)
- উদাহরণ: গড় নম্বর বের করা।

৪. কোয়েরি করা (Query using SQL)

- SQL ব্যবহার করে নির্দিষ্ট তথ্য বের করা হয়।
- উদাহরণ:
- `SELECT AVG(Marks) FROM Student;`

৫. ডাটা বিশ্লেষণ (Data Analysis)

- ডাটার উপর বিশ্লেষণ করে গুরুত্বপূর্ণ তথ্য বের করা হয়।
- উদাহরণ: কে টপার, কতজন পাস করেছে।

৬. তথ্য প্রদর্শন (Information Output)

- শেষে রিপোর্ট, চার্ট বা সারাংশ আকারে দেখানো হয়।
- এটিই অর্থপূর্ণ তথ্য (Information)।

📌 সহজ উদাহরণ

- ডাটা: ৫০, ৬০, ৭০
- তথ্য: “গড় নম্বর = ৬০”

📁 সংক্ষেপে

👉 ডাটা → সংরক্ষণ → প্রক্রিয়াকরণ → কোয়েরি → বিশ্লেষণ → তথ্য

1.3 What is database and database management system

✅ Answer in English

♦ What is a Database?

A **database** is an **organized collection of related data** that is stored in a structured way so it can be easily accessed, managed, and updated.

- It stores data in the form of **tables** (rows and columns).
- Data is related and meaningful.
- It helps in efficient data storage and retrieval.

📌 Example:

A student database may contain:

- ID
- Name

- Department
- Marks

👉 So, a database is basically **where data is stored in an organized way.**

♦ What is a Database Management System (DBMS)?

A **Database Management System (DBMS)** is a **software** that allows users to **create, manage, and interact with databases.**

- It acts as an **interface between user and database.**
- It helps in:
 - Inserting data
 - Updating data
 - Deleting data
 - Retrieving data

📌 Examples of DBMS:

- MySQL
- Oracle
- SQL Server

📁 Key Differences

Aspect	Database	DBMS
Definition	Collection of data	Software to manage database
Role	Stores data	Manages and controls data
Nature	Passive (just data)	Active (performs operations)
Example	Student table	MySQL, Oracle

📌 Simple Understanding

- **Database** = Data storage
- **DBMS** = Tool/software to handle that data

✅ বাংলা উত্তর (Bangla)

♦ ডাটাবেজ (Database) কী?

ডাটাবেজ হলো **সম্পর্কিত ডাটার একটি সংগঠিত সংগ্রহ**, যা এমনভাবে সংরক্ষণ করা হয় যাতে সহজে ব্যবহার ও পরিবর্তন করা যায়।

- ডাটা টেবিল আকারে (row ও column) রাখা হয়
- ডাটা পরস্পরের সাথে সম্পর্কযুক্ত থাকে
- সহজে খুঁজে পাওয়া যায়

📌 উদাহরণ:

একটি Student Database এ থাকতে পারে:

- ID
- Name
- Department

- Marks

👉 অর্থাৎ, ডাটাবেজ হলো **ডাটা সংরক্ষণের একটি সংগঠিত ব্যবস্থা**।

♦ ডাটাবেজ ম্যানেজমেন্ট সিস্টেম (DBMS) কী?

DBMS হলো একটি **সফটওয়্যার**, যা ডাটাবেজ তৈরি, নিয়ন্ত্রণ এবং ব্যবহার করতে সাহায্য করে।

- এটি ব্যবহারকারী এবং ডাটাবেজের মধ্যে সংযোগ তৈরি করে
- এর মাধ্যমে আমরা:
 - ডাটা যোগ করতে পারি (Insert)
 - পরিবর্তন করতে পারি (Update)
 - মুছে ফেলতে পারি (Delete)
 - খুঁজে বের করতে পারি (Retrieve)

🔴 উদাহরণ:

- MySQL
- Oracle
- SQL Server

📁 মূল পার্থক্য

বিষয়	ডাটাবেজ (Database)	DBMS
সংজ্ঞা	ডাটার সংগ্রহ	ডাটা পরিচালনার সফটওয়্যার
কাজ	ডাটা সংরক্ষণ	ডাটা নিয়ন্ত্রণ ও ব্যবস্থাপনা
প্রকৃতি	স্থির (Passive)	সক্রিয় (Active)
উদাহরণ	Student Table	MySQL, Oracle

🔴 সহজভাবে বুঝলে

- Database = ডাটা রাখার জায়গা
- DBMS = সেই ডাটা পরিচালনার সফটওয়্যার

1.4 What is the main role and advantage of DBMS?

✅ Answer in English

♦ Main Role of DBMS (Database Management System)

The main role of a DBMS is to **manage and control data efficiently** in a database. It acts as a bridge between users and the database.

Key Roles:

1. **Data Storage & Organization**
 - Stores data in a structured way (tables, rows, columns).
2. **Data Retrieval**
 - Allows users to easily retrieve data using queries (SQL).

3. Data Manipulation

- Supports operations like **Insert, Update, Delete**.

4. Data Security

- Protects data from unauthorized access using authentication and permissions.

5. Data Integrity

- Ensures data is accurate and consistent.

6. Concurrency Control

- Allows multiple users to access data at the same time without conflict.

7. Backup & Recovery

- Provides backup and restores data in case of failure.

◆ Advantages of DBMS

1. Reduced Data Redundancy

- Avoids duplicate data.

2. Improved Data Consistency

- Same data remains consistent across the system.

3. Better Data Security

- Only authorized users can access or modify data.

4. Easy Data Access

- Data can be easily retrieved using queries.

5. Data Sharing

- Multiple users can access the same data.

6. Data Integrity

- Maintains correctness of data.

7. Backup & Recovery

- Prevents data loss.

✦ Simple Summary

👉 **Role** = Manage and control data

👉 **Advantage** = Makes data safe, consistent, and easy to use

✅ বাংলা উত্তর (Bangla)

◆ DBMS এর প্রধান ভূমিকা (Main Role)

DBMS এর প্রধান কাজ হলো **ডাটাকে সঠিকভাবে সংরক্ষণ, নিয়ন্ত্রণ এবং ব্যবস্থাপনা করা।**

প্রধান কাজগুলো:

1. ডাটা সংরক্ষণ ও সংগঠন

- টেবিল আকারে ডাটা সংরক্ষণ করে।

2. ডাটা বের করা (Retrieval)

- SQL ব্যবহার করে সহজে ডাটা বের করা যায়।

3. ডাটা পরিবর্তন (Manipulation)

- Insert, Update, Delete করা যায়।

4. ডাটা নিরাপত্তা (Security)

- অনুমতি ছাড়া কেউ ডাটা ব্যবহার করতে পারে না।

5. ডাটার সঠিকতা (Integrity)

- ডাটা সঠিক ও নির্ভুল থাকে।

6. একাধিক ব্যবহারকারী (Concurrency)

- অনেক user একসাথে ডাটা ব্যবহার করতে পারে।
7. ব্যাকআপ ও রিকভারি
- ডাটা নষ্ট হলে পুনরুদ্ধার করা যায়।

◆ DBMS এর সুবিধা (Advantages)

1. ডাটা পুনরাবৃত্তি কমায় (Redundancy কমায়)
2. ডাটার সামঞ্জস্য বজায় রাখে (Consistency)
3. ডাটা নিরাপদ রাখে (Security)
4. সহজে ডাটা পাওয়া যায়
5. একাধিক user ডাটা ব্যবহার করতে পারে
6. ডাটা সঠিক থাকে (Integrity)
7. ব্যাকআপ ও রিকভারি সুবিধা দেয়

✚ সহজভাবে

👉 Role = ডাটা নিয়ন্ত্রণ করা

👉 Advantage = ডাটাকে নিরাপদ, সঠিক ও সহজ ব্যবহারযোগ্য করা

1.5 Database application.

◆ What is a Database Application?

A **database application** is a software program that **uses a database to store, manage, and retrieve data** for specific tasks.

👉 It connects the **user interface (UI)** with the **database** through a DBMS.

◆ Where Database Applications are Used (Applications/Fields)

Database applications are used in many real-life systems:

1. Banking System

- Stores customer details, account info, transactions.
- Example: ATM, online banking.

2. Education System

- Manages student records, results, attendance.
- Example: School/University management systems.

3. Hospital Management System

- Stores patient records, doctor info, appointments.
- Helps in diagnosis and treatment tracking.

4. E-Commerce (Online Shopping)

- Manages products, customers, orders, payments.
- Example: Amazon, Daraz.

5. Airline/Railway Reservation System

- Handles ticket booking, schedules, passenger info.

6. Telecommunication

- Stores call records, billing information.

7. Library Management System

- Tracks books, members, issue/return records.

◆ Key Features of Database Applications

- Data storage and retrieval
- User-friendly interface
- Fast processing
- Multi-user support

✦ Simple Summary

👉 Database Application = **Software + Database + DBMS**

👉 Used to **solve real-life problems using data**

✅ বাংলা উত্তর (Bangla)

◆ ডাটাবেজ অ্যাপ্লিকেশন কী?

ডাটাবেজ অ্যাপ্লিকেশন হলো এমন একটি **সফটওয়্যার**, যা ডাটাবেজ ব্যবহার করে **ডাটা সংরক্ষণ, পরিচালনা এবং ব্যবহার করে**।

👉 এটি **User Interface** এবং **Database**-এর মধ্যে সংযোগ তৈরি করে।

◆ ডাটাবেজ অ্যাপ্লিকেশনের ব্যবহার ক্ষেত্র

১. ব্যাংকিং সিস্টেম

- গ্রাহকের তথ্য, লেনদেন সংরক্ষণ করে।
- উদাহরণ: ATM, Online Banking

২. শিক্ষা ব্যবস্থা

- শিক্ষার্থীর তথ্য, ফলাফল, উপস্থিতি সংরক্ষণ করে।

৩. হাসপাতাল ম্যানেজমেন্ট

- রোগীর তথ্য, ডাক্তার, অ্যাপয়েন্টমেন্ট সংরক্ষণ করে।

৪. ই-কমার্স (অনলাইন শপিং)

- পণ্য, অর্ডার, কাস্টমার তথ্য পরিচালনা করে।
- উদাহরণ: Amazon, Daraz

৫. টিকিট বুকিং সিস্টেম

- বিমান/ট্রেন টিকিট বুকিং পরিচালনা করে।

৬. টেলিকম

- কল রেকর্ড ও বিলিং তথ্য সংরক্ষণ করে।

৭. লাইব্রেরি ম্যানেজমেন্ট

- বই, সদস্য, ইস্যু/রিটার্ন ট্র্যাক করে।

◆ মূল বৈশিষ্ট্য

- ডাটা সংরক্ষণ ও বের করা
- সহজ ব্যবহারযোগ্য ইন্টারফেস
- দ্রুত কাজ করে
- একাধিক ব্যবহারকারী সাপোর্ট

📌 সহজভাবে

👉 ডাটাবেজ অ্যাপ্লিকেশন = ডাটাবেজ ব্যবহার করে তৈরি সফটওয়্যার

👉 বাস্তব জীবনের সমস্যা সমাধানে ব্যবহৃত হয়

1.6 How DBMS manages the interaction between the end user and the database

◆ How DBMS Manages Interaction Between End User and Database

A **DBMS (Database Management System)** acts as a **middle layer (interface)** between the **end user** and the **database**. It hides the complexity of the database and allows users to interact easily.

📋 Step-by-Step Process

1. User Request (Input)

- The end user gives a request through an **application** or directly using **SQL**.
- Example: "Show all student marks"

2. Query Processing

- The DBMS receives the request and **converts it into a query**.
- It checks the syntax and understands what data is needed.

3. Query Optimization

- DBMS decides the **best and fastest way** to retrieve the data.

4. Execution

- The DBMS accesses the database and **executes the query**.
- It reads or modifies data as requested.

5. Data Retrieval

- The required data is collected from the database tables.

6. Result Presentation (Output)

- The DBMS sends the result back to the user in a **readable format** (table, report, etc.).

◆ Additional Controls by DBMS

- **Security Check** → Verifies user permission
- **Integrity Check** → Ensures correct data
- **Concurrency Control** → Handles multiple users

- **Backup & Recovery** → Protects data

✦ Simple Flow

👉 User → DBMS → Database → DBMS → User

✦ Example

User types:

SELECT * FROM Student;

👉 DBMS processes it and shows all student records.

✅ বাংলা উত্তর (Bangla)

- ◆ **DBMS কীভাবে User এবং Database-এর মধ্যে যোগাযোগ পরিচালনা করে**

DBMS একটি **মধ্যবর্তী স্তর (interface)** হিসেবে কাজ করে, যা **ব্যবহারকারী (end user)** এবং **ডাটাবেজ**-এর মধ্যে সংযোগ তৈরি করে।

🔄 ধাপে ধাপে প্রক্রিয়া

১. User Request (ইনপুট)

- ব্যবহারকারী অ্যাপ্লিকেশন বা SQL এর মাধ্যমে অনুরোধ করে।
- উদাহরণ: “সব ছাত্রের নম্বর দেখাও”

২. Query Processing

- DBMS অনুরোধ গ্রহণ করে এবং সেটিকে query-তে রূপান্তর করে।
- এটি syntax ঠিক আছে কিনা তা চেক করে।

৩. Query Optimization

- DBMS সবচেয়ে দ্রুত উপায় নির্বাচন করে ডাটা বের করার জন্য।

৪. Execution (কার্য সম্পাদন)

- DBMS ডাটাবেজে গিয়ে query চালায়।

৫. Data Retrieval

- প্রয়োজনীয় ডাটা টেবিল থেকে বের করা হয়।

৬. Result Output

- DBMS ব্যবহারকারীর কাছে ফলাফল দেখায় (table/report আকারে)।

◆ অতিরিক্ত নিয়ন্ত্রণ

- **Security** → অনুমতি যাচাই
- **Integrity** → ডাটার সঠিকতা নিশ্চিত
- **Concurrency** → একাধিক user handle
- **Backup & Recovery** → ডাটা সুরক্ষা

✦ সহজ ফ্লো

👉 User → DBMS → Database → DBMS → User

📌 উদাহরণ

SELECT * FROM Student;

👉 DBMS সব student এর ডাটা দেখায়।

1.7 types of database in different things.

Databases can be classified into different types based on various criteria such as data structure, usage, and environment. The main types are:

1. Based on Data Model

- **Hierarchical Database**

Data is organized in a tree-like structure (parent-child relationship). Each child has only one parent.

Example: IBM IMS

- **Network Database**

Data is represented as a graph where a record can have multiple parent and child relationships (many-to-many).

- **Relational Database (RDBMS)**

Data is stored in tables (rows and columns). Relationships are maintained using keys.

Example: MySQL, Oracle, PostgreSQL

- **Object-Oriented Database (OODBMS)**

Data is stored in the form of objects, similar to object-oriented programming concepts.

- **NoSQL Database**

Designed for unstructured or semi-structured data. It includes:

- Document-based (e.g., JSON)
- Key-value stores
- Column-based
- Graph databases

Example: MongoDB, Cassandra

2. Based on Usage / Environment

- **Centralized Database**

Stored at a single location and accessed by users from different locations.

- **Distributed Database**

Data is distributed across multiple locations but managed as a single system.

- **Cloud Database**

Hosted on cloud platforms and accessed via the internet.

3. Based on User Access

- **Single-user Database**

Supports one user at a time.

- **Multi-user Database**

Supports multiple users simultaneously.

4. Based on Purpose

- **Operational Database (OLTP)**

Used for day-to-day transactions.

- **Analytical Database (OLAP)**
Used for data analysis and decision-making.

Answer (Bangla)

ডাটাবেস বিভিন্ন ভিত্তিতে বিভিন্ন ধরনের হতে পারে, যেমন ডেটার গঠন, ব্যবহার এবং পরিবেশ অনুযায়ী। প্রধান ধরনেরগুলো হলো:

1. Data Model অনুযায়ী

- **Hierarchical Database**
ডেটা ট্রি (tree) আকারে সাজানো থাকে (parent-child সম্পর্ক)। প্রতিটি child-এর একটাই parent থাকে।
- **Network Database**
গ্রাফ আকারে ডেটা থাকে যেখানে একাধিক parent-child সম্পর্ক (many-to-many) সম্ভব।
- **Relational Database (RDBMS)**
ডেটা টেবিল আকারে (row ও column) সংরক্ষিত হয় এবং key ব্যবহার করে সম্পর্ক তৈরি করা হয়।
উদাহরণ: MySQL, Oracle, PostgreSQL
- **Object-Oriented Database (OODBMS)**
ডেটা object আকারে সংরক্ষিত হয়, যেমন object-oriented programming এর মতো।
- **NoSQL Database**
Unstructured বা semi-structured ডেটার জন্য ব্যবহৃত হয়। এর মধ্যে আছে:
 - Document-based
 - Key-value store
 - Column-based
 - Graph databaseউদাহরণ: MongoDB, Cassandra

2. Usage / Environment অনুযায়ী

- **Centralized Database**
একটি নির্দিষ্ট স্থানে ডেটা থাকে এবং বিভিন্ন জায়গা থেকে access করা যায়।
- **Distributed Database**
ডেটা একাধিক স্থানে ছড়িয়ে থাকে কিন্তু একটি সিস্টেম হিসেবে কাজ করে।
- **Cloud Database**
ক্লাউডে হোস্ট করা হয় এবং ইন্টারনেটের মাধ্যমে access করা যায়।

3. User Access অনুযায়ী

- **Single-user Database**
এক সময়ে একজন user ব্যবহার করতে পারে।
- **Multi-user Database**
একাধিক user একই সাথে ব্যবহার করতে পারে।

4. Purpose অনুযায়ী

- **Operational Database (OLTP)**
দৈনন্দিন transaction এর জন্য ব্যবহৃত হয়।
- **Analytical Database (OLAP)**
data analysis ও decision making এর জন্য ব্যবহৃত হয়।

1.8 purpose of database system.

The **purpose of a database system** is to efficiently store, manage, and retrieve data while ensuring data integrity, security, and consistency. The main purposes are:

1. Data Storage and Organization

A database system provides a structured way to store large amounts of data so that it can be easily accessed and managed.

2. Efficient Data Retrieval

It allows users to quickly search, query, and retrieve required information using query languages like SQL.

3. Data Sharing

Multiple users and applications can access the same data simultaneously in a controlled manner.

4. Data Integrity

Ensures that the data remains accurate and consistent by applying constraints and rules.

5. Data Security

Protects data from unauthorized access using authentication, authorization, and access control mechanisms.

6. Data Redundancy Reduction

Minimizes duplicate data by centralizing storage and using normalization techniques.

7. Data Independence

Separates data from the application programs so that changes in data structure do not affect applications.

8. Backup and Recovery

Provides mechanisms to backup data and recover it in case of system failure or crashes.

Answer (Bangla)

ডাটাবেস সিস্টেমের মূল উদ্দেশ্য হলো ডেটা দক্ষভাবে সংরক্ষণ, পরিচালনা এবং প্রয়োজনে দ্রুত পুনরুদ্ধার করা, পাশাপাশি ডেটার নিরাপত্তা, নির্ভুলতা এবং সামঞ্জস্য বজায় রাখা। প্রধান উদ্দেশ্যগুলো হলো:

1. Data Storage and Organization

বড় পরিমাণ ডেটা সুশৃঙ্খলভাবে সংরক্ষণ করা যাতে সহজে access করা যায়।

2. Efficient Data Retrieval

SQL এর মতো query language ব্যবহার করে দ্রুত প্রয়োজনীয় ডেটা খুঁজে বের করা।

3. Data Sharing

একাধিক user ও application একই ডেটা একসাথে ব্যবহার করতে পারে।

4. Data Integrity

ডেটার সঠিকতা ও সামঞ্জস্য বজায় রাখে এবং rule/constraint প্রয়োগ করে।

5. Data Security

Unauthorized access থেকে ডেটাকে রক্ষা করে (authentication ও authorization ব্যবহার করে)।

6. Data Redundancy Reduction

অপ্রয়োজনীয় duplicate ডেটা কমিয়ে আনে।

7. Data Independence

Application program থেকে data আলাদা রাখে, ফলে structure পরিবর্তন হলেও application প্রভাবিত হয় না।

8. Backup and Recovery

ডেটার backup রাখা হয় এবং system failure হলে recovery করা যায়।

1.9 why database design is important.

Answer (English)

Database design is important because it ensures that data is stored efficiently, accurately, and in a way that supports easy access and management. The main reasons are:

1. Improves Data Organization

A good design structures data properly into tables, making it easy to understand and manage.

2. Reduces Data Redundancy

Proper design (using normalization) minimizes duplicate data, saving storage space and avoiding inconsistency.

3. Maintains Data Integrity

Ensures accuracy and consistency of data by applying constraints such as primary keys, foreign keys, and rules.

4. Enhances Performance

Well-designed databases allow faster queries and efficient data retrieval.

5. Supports Data Security

Helps in defining access control and permissions to protect sensitive data.

6. Makes Maintenance Easier

A structured design simplifies updates, modifications, and troubleshooting.

7. Ensures Scalability

A good design allows the database to grow with increasing data and users without major issues.

8. Avoids Data Anomalies

Proper design prevents insertion, update, and deletion anomalies.

Answer (Bangla)

ডাটাবেস ডিজাইন গুরুত্বপূর্ণ কারণ এটি ডেটাকে সঠিকভাবে, দক্ষভাবে এবং সুশৃঙ্খলভাবে সংরক্ষণ ও ব্যবস্থাপনা করতে সাহায্য করে। প্রধান কারণগুলো হলো:

1. Data Organization উন্নত করে

সঠিক ডিজাইন ডেটাকে টেবিল আকারে সুন্দরভাবে সাজায়, ফলে বোঝা ও ব্যবস্থাপনা সহজ হয়।

2. Data Redundancy কমায়

Normalization এর মাধ্যমে duplicate ডেটা কমানো যায়, ফলে storage সাশ্রয় হয় এবং inconsistency কমে।

3. Data Integrity বজায় রাখে

Primary key, foreign key ইত্যাদি constraint ব্যবহার করে ডেটার সঠিকতা ও সামঞ্জস্য নিশ্চিত করে।

4. Performance উন্নত করে

ভাল ডিজাইন দ্রুত query execution এবং efficient data retrieval নিশ্চিত করে।

5. Data Security নিশ্চিত করে

Access control ও permission সেট করে sensitive ডেটা রক্ষা করে।

6. Maintenance সহজ করে

Structured design থাকার কারণে update, modification এবং troubleshooting সহজ হয়।

7. Scalability নিশ্চিত করে

ডেটা ও user বাড়লেও system সহজে expand করা যায়।

8. Data Anomalies এড়ায়

Insertion, update ও deletion anomaly প্রতিরোধ করে।

1.10 Environment of database management system.

◆ Environment of Database Management System (DBMS)

The **DBMS environment** consists of all the components that interact to manage, store, retrieve, and control data in a database system.

◆ Main Components of DBMS Environment

◆ 1. Hardware

- Physical devices where the database is stored.
- Includes servers, computers, storage devices, network devices.

👉 Example: Hard disk, SSD, cloud servers.

◆ 2. Software

- The DBMS software itself and related applications.
- Includes:
 - DBMS (e.g., MS SQL Server, Oracle)
 - Operating system
 - Application programs

◆ 3. Data

- The actual stored information in the database.
- Includes both:
 - Operational data
 - Metadata (data about data)

◆ 4. Users (People)

Different types of users interact with the DBMS:

- **Database Administrator (DBA)** → controls and manages the database
- **Application Programmers** → write programs using the database
- **End Users** → use applications to access data

◆ 5. Procedures

- Rules and instructions for managing the database.
- Includes guidelines for:
 - Data access
 - Backup
 - Security
 - Recovery

◆ Working of DBMS Environment

- Users interact with applications
- Applications communicate with DBMS software
- DBMS processes queries and accesses data from storage
- Hardware stores and retrieves the data

📁 Simple Summary

👉 DBMS environment = Hardware + Software + Data + Users + Procedures working together

✅ বাংলা উত্তর (Bangla)

◆ DBMS Environment কী?

DBMS environment হলো এমন একটি পরিবেশ যেখানে hardware, software, data, user এবং procedure একসাথে কাজ করে database পরিচালনা করে।

◆ DBMS Environment-এর উপাদানসমূহ

◆ ১. Hardware

- Database যেখানে রাখা হয় সেই physical devices
- যেমন: computer, server, hard disk

◆ ২. Software

- DBMS software এবং অন্যান্য related software
- যেমন: MS SQL Server, Oracle, OS, application software

◆ ৩. Data

- Database-এর ভিতরে থাকা আসল তথ্য
- যেমন: student data, customer data
- Metadata (data সম্পর্কে data) ও অন্তর্ভুক্ত

◆ ৪. Users (ব্যবহারকারী)

- **DBA (Database Administrator)** → database control করে
- **Programmer** → application তৈরি করে
- **End User** → data ব্যবহার করে

◆ ৫. Procedures

- Database ব্যবস্থাপনার নিয়ম ও নির্দেশনা
- যেমন: backup, security, recovery rules

◆ DBMS Environment কিভাবে কাজ করে

- User application ব্যবহার করে
- Application DBMS-এর সাথে যোগাযোগ করে
- DBMS data process করে
- Hardware data store ও retrieve করে

1.11 Function of database.

◆ Functions of a Database

A database is designed to store, manage, and retrieve data efficiently. Its main functions include:

◆ 1. Data Storage

- Stores large amounts of data in an organized way.
- Data is stored in tables (rows and columns).

◆ 2. Data Retrieval

- Allows users to quickly search and access required data.
- Uses queries (SQL) to fetch information.

◆ 3. Data Manipulation

- Supports inserting, updating, and deleting data.
- Keeps data up to date.

◆ 4. Data Security

- Protects data from unauthorized access.
- Uses authentication, authorization, and permissions.

◆ 5. Data Integrity

- Ensures accuracy and consistency of data.
- Uses constraints like primary key, foreign key, etc.

◆ 6. Data Sharing

- Multiple users can access the same database simultaneously.
- Supports multi-user environment.

◆ 7. Backup and Recovery

- Provides mechanisms to back up data.
- Helps restore data in case of system failure.

◆ 8. Concurrency Control

- Manages simultaneous access by multiple users.
- Prevents conflicts when many users use the database at the same time.

🔄 Simple Summary

👉 Database functions include storing, retrieving, updating, securing, and managing data efficiently.

◆ Database-এর Function (কার্যাবলী)

Database-এর মূল কাজ হলো data সংরক্ষণ, পরিচালনা এবং সহজে ব্যবহারযোগ্য করে তোলা।

◆ ১. Data Storage (ডাটা সংরক্ষণ)

- বড় পরিমাণ data organized ভাবে সংরক্ষণ করে
- Table আকারে data রাখা হয়

◆ ২. Data Retrieval (ডাটা খুঁজে বের করা)

- দ্রুত data search ও access করা যায়
- SQL query ব্যবহার করা হয়

◆ ৩. Data Manipulation (ডাটা পরিবর্তন)

- Insert, Update, Delete করা যায়
- Data আপডেট রাখা যায়

◆ ৪. Data Security (নিরাপত্তা)

- Unauthorized access থেকে data রক্ষা করে
- Username/password, permission ব্যবহার করা হয়

◆ ৫. Data Integrity (সঠিকতা)

- Data যেন accurate ও consistent থাকে তা নিশ্চিত করে
- Primary key, foreign key ইত্যাদি constraints ব্যবহার করা হয়

◆ ৬. Data Sharing (শেয়ারিং)

- একাধিক user একই database ব্যবহার করতে পারে

◆ ৭. Backup & Recovery

- Data backup রাখা যায়
- System failure হলে data restore করা যায়

◆ ৮. Concurrency Control

- একাধিক user একই সময়ে কাজ করলে সমস্যা নিয়ন্ত্রণ করে

📦 সহজভাবে

👉 Database data store, retrieve, update, secure এবং manage করার কাজ করে

1.12 Abstraction of database management system.

◆ Abstraction in Database Management System (DBMS)

Database abstraction means hiding the complex details of how data is stored and managed, and showing only the necessary information to the user.

👉 It allows users to interact with the database without knowing internal implementation details.

◆ Levels of Data Abstraction

DBMS is divided into **three levels**:

◆ 1. Physical Level (Internal Level)

- The lowest level of abstraction
- Describes **how data is physically stored**
- Includes storage structures, file organization, indexes

👉 Example: How data is stored on disk or memory

◆ 2. Logical Level (Conceptual Level)

- Describes **what data is stored and relationships among data**
- Defines tables, attributes, relationships
- Used by database designers and administrators

👉 Example: Student table with ID, Name, Age

◆ 3. View Level (External Level)

- The highest level of abstraction
- Shows only a **part of the database** to the user
- Different users can have different views

👉 Example: A student sees only their own marks, not the whole database

◆ Advantages of Abstraction

- Simplifies database usage
- Hides complexity
- Improves security
- Provides flexibility in data handling

📄 Simple Summary

👉 Database abstraction = hiding internal details and showing only necessary data through 3 levels: Physical, Logical, View.

✅ বাংলা উত্তর (Bangla)

◆ DBMS-এ Abstraction কী?

Database abstraction হলো এমন একটি ধারণা যেখানে database-এর complex ভিতরের কাজগুলো লুকিয়ে রেখে user-কে শুধু প্রয়োজনীয় তথ্য দেখানো হয়।

👉 এতে user database-এর ভিতরের structure না জেনেও কাজ করতে পারে।

◆ Database Abstraction-এর ৩টি স্তর

◆ ১. Physical Level (Internal Level)

- সবচেয়ে নিচের স্তর
- Data কিভাবে storage-এ রাখা হয় তা দেখায়
- Disk, memory, file structure ইত্যাদি অন্তর্ভুক্ত

👉 উদাহরণ: Data disk-এ কিভাবে store হচ্ছে

♦ ২. Logical Level (Conceptual Level)

- Database-এর structure কীভাবে organized তা দেখায়
- Table, attribute, relationship নির্ধারণ করে
- DBA ও designer ব্যবহার করে

👉 উদাহরণ: Student table (ID, Name, Age)

♦ ৩. View Level (External Level)

- User কে database-এর একটি অংশ দেখায়
- বিভিন্ন user বিভিন্ন view পায়

👉 উদাহরণ: একজন student শুধু নিজের result দেখে

♦ Abstraction-এর সুবিধা

- সহজে ব্যবহার করা যায়
- Complexity hide থাকে
- Security বাড়ে
- Flexible data management সম্ভব

📦 সহজভাবে

👉 Database abstraction = 3 level (Physical, Logical, View) দিয়ে complex data hide করে user-friendly access প্রদান করা

2.0 What is metadata?

♦ What is Metadata?

Metadata is data about data.

👉 It describes the **structure, type, and properties of data** stored in a database, rather than the actual data itself.

♦ Examples of Metadata

- Table name
- Column names (fields)
- Data types (INT, VARCHAR, DATE)
- Constraints (PRIMARY KEY, FOREIGN KEY)
- Relationships between tables

📌 Example:

Student Table:

- ID → INT
- Name → VARCHAR
- Age → INT

👉 These descriptions are **metadata**, while the actual records are data.

♦ Types of Metadata

1. **Descriptive Metadata** → describes the content (e.g., title, name)

2. **Structural Metadata** → describes how data is organized (tables, relationships)
3. **Administrative Metadata** → information about data management (permissions, creation date)

◆ Importance of Metadata

- Helps understand data structure
- Assists in database design
- Improves data management
- Useful for querying and maintaining databases

📄 Simple Summary

👉 Metadata = Information that describes data (data about data)

✅ বাংলা উত্তর (Bangla)

◆ Metadata কী?

Metadata হলো data সম্পর্কে data।

👉 এটি মূল data নয়, বরং data-এর structure, type এবং properties সম্পর্কে তথ্য দেয়।

◆ Metadata-এর উদাহরণ

- Table name
- Column name
- Data type (INT, VARCHAR, DATE)
- Primary key, foreign key
- Table-এর relationship

📌 উদাহরণ:

Student Table:

- ID → INT
- Name → VARCHAR
- Age → INT

👉 এগুলো metadata, আর actual student records হলো data

◆ Metadata-এর ধরন

1. **Descriptive Metadata** → data-এর description
2. **Structural Metadata** → data কিভাবে organized তা দেখায়
3. **Administrative Metadata** → permission, creation date ইত্যাদি

◆ Metadata-এর গুরুত্ব

- Data structure বোঝাতে সাহায্য করে
- Database design সহজ করে
- Data management উন্নত করে
- Query করার জন্য দরকার হয়

📄 সহজভাবে

👉 Metadata = data সম্পর্কে তথ্য (data about data)

2.1 Historical roots of file and file system in DBMS.

Answer (English)

The historical roots of file and file systems explain how data management evolved before the introduction of DBMS.

1. Manual File Systems

- In early days, data was stored in **physical file folders** kept in cabinets.
- Organization of data was based on expected usage and logical grouping.
- This system worked for small amounts of data but became **time-consuming and inefficient** as data grew.

2. Transition to Computer-Based File Systems

- With the development of computers, manual file systems were converted into **computerized file systems**.
- Initially, computer files were designed similarly to manual files.
- Data was stored in separate files for different purposes, and each department managed its own data.

3. Application-Specific Files

- Each application (e.g., sales, payroll, inventory) had its own files and programs.
- For example:
 - Sales department had a sales database
 - Personnel department had an agent/employee database
- Each file was owned and controlled by the department that created it.

4. Role of DP Specialists

- Data Processing (DP) specialists wrote programs to generate reports such as:
 - Monthly sales summaries
 - Customer renewal reports
 - Analysis reports and letters
- This required writing separate programs for each new task.

5. Growth and Fragmentation of File Systems

- As organizations grew, multiple independent file systems were created.
- Each file had its own structure and application programs.
- This led to duplication, lack of coordination, and difficulty in data management.

6. Problems in File-Based Systems

From the slide, key issues included:

- Data redundancy and inconsistency
- Multiple file formats
- Difficulty in accessing data
- Need to write new programs for each task
- Integrity and security problems
- Issues with concurrent access and updates
- Atomicity problems (partial updates causing inconsistency)

7. Emergence of DBMS

- Due to these limitations, DBMS was introduced.
- DBMS provides centralized control, reduces redundancy, ensures consistency, improves security, and supports concurrent access.

Answer (Bangla)

DBMS-এর file system এর historical root বোঝায় কিভাবে DBMS আসার আগে data management করা হতো।

1. Manual File System

- শুরুতে ডেটা কাগজের ফোল্ডারে রাখা হতো (file cabinet এ)।
- ডেটা logicalভাবে সাজানো থাকত এবং ব্যবহার অনুযায়ী organize করা হতো।
- ছোট ডেটার জন্য এটি কার্যকর হলেও বড় ডেটার ক্ষেত্রে অদক্ষ হয়ে পড়ে।

2. Computer-Based File System

- কম্পিউটার আসার পর manual file system কে digital করা হয়।
- প্রথমদিকে computer file system manual system এর মতোই ডিজাইন করা হয়েছিল।
- বিভিন্ন data আলাদা আলাদা file এ সংরক্ষণ করা হতো।

3. Application Specific File

- প্রতিটি department (sales, personnel ইত্যাদি) নিজের আলাদা database/file ব্যবহার করত।
- প্রতিটি file তার own department দ্বারা control করা হতো।

4. DP Specialist এর ভূমিকা

- Data Processing (DP) specialist বিভিন্ন report তৈরি করার জন্য program লিখত:
 - Monthly sales report
 - Customer renewal report
 - Analysis report
- প্রতিটি কাজের জন্য আলাদা program দরকার হতো।

5. File System এর বৃদ্ধি ও সমস্যা

- Organization বড় হওয়ার সাথে সাথে অনেক independent file system তৈরি হয়।
- প্রতিটি file এর আলাদা structure ও program থাকত।
- এতে data management জটিল হয়ে যায়।

6. File System এর সমস্যা

Slide অনুযায়ী প্রধান সমস্যাগুলো:

- Data redundancy ও inconsistency
- Multiple file format
- Data access করা কঠিন
- নতুন কাজের জন্য নতুন program লিখতে হয়
- Integrity ও security সমস্যা
- Concurrent access সমস্যা
- Atomicity সমস্যা (partial update)

7. DBMS এর উদ্ভব

- এসব সমস্যার কারণে DBMS তৈরি হয়।
- DBMS centralized control, data sharing, consistency, security এবং efficient access নিশ্চিত করে।

❖ Disadvantages of Traditional File System

Traditional file systems are the older way of storing and managing data using files without a DBMS. They have several limitations:

◆ 1. Data Redundancy

- Same data is stored in multiple files.
- Leads to duplication and waste of storage.

👉 **Example: Student name stored in multiple department files.**

◆ 2. Data Inconsistency

- Because of redundancy, data may not be the same in all places.
- Updating one file but not others causes inconsistency.

◆ 3. Data Isolation

- Data is scattered across different files and formats.
- Difficult to access and combine data.

◆ 4. Difficulty in Data Access

- No efficient query system.
- Requires writing complex programs to retrieve data.

◆ 5. Lack of Data Integrity

- No proper constraints to ensure accuracy of data.
- Errors and invalid data may exist.

◆ 6. Security Issues

- Limited control over who can access data.
- Data is more vulnerable to unauthorized access.

◆ 7. Data Dependence

- Programs are tightly dependent on file structure.
- Any change in file format requires modification of programs.

◆ 8. No Concurrent Access Control

- Multiple users accessing the same data may cause conflicts.
- No proper mechanism to handle simultaneous access.

◆ 9. Backup and Recovery

- No automatic backup and recovery system.
- Data loss can be critical.

Simple Summary

 Traditional file system suffers from redundancy, inconsistency, poor security, and difficulty in managing data.

✅ বাংলা উত্তর (Bangla)

◆ Traditional File System-এর অসুবিধা

Traditional file system হলো DBMS ছাড়া ফাইলের মাধ্যমে data সংরক্ষণের পদ্ধতি। এর অনেক সীমাবদ্ধতা আছে:

◆ ১. Data Redundancy (ডুপ্লিকেশন)

- একই data বারবার বিভিন্ন file-এ রাখা হয়
- Storage waste হয়

 উদাহরণ: একই student name বিভিন্ন file-এ থাকতে পারে

♦ ২. Data Inconsistency (অসঙ্গতি)

- একই data বিভিন্ন file-এ আলাদা হতে পারে
- এক জায়গায় update করলে অন্য জায়গায় না হলে inconsistency হয়

♦ ৩. Data Isolation

- Data বিভিন্ন file-এ ছড়িয়ে থাকে
- একসাথে access করা কঠিন

♦ ৪. Data Access করা কঠিন

- Efficient query system নেই
- Data পেতে complex program লিখতে হয়

♦ ৫. Data Integrity সমস্যা

- Data validation/constraint ঠিকভাবে থাকে না
- ভুল data থাকার সম্ভাবনা থাকে

♦ ৬. Security সমস্যা

- Access control দুর্বল
- Unauthorized access হতে পারে

♦ ৭. Program Dependency

- File structure পরিবর্তন হলে program পরিবর্তন করতে হয়

♦ ৮. Concurrent Access সমস্যা

- একাধিক user একসাথে কাজ করলে conflict হতে পারে

♦ ৯. Backup ও Recovery সমস্যা

- Automatic backup system নেই
- Data loss হলে recovery কঠিন

 সহজভাবে

 Traditional file system এ redundancy, inconsistency, security সমস্যা এবং data management কঠিন

2.2 What is data?

 Answer in English

Data refers to **raw facts and figures** that have no meaning on their own.

- It is **unprocessed and unorganized**.
- Data can be in the form of **numbers, text, symbols, or observations**.
- It does not provide useful information until it is processed.

 Examples:

- Numbers: 10, 20, 30

- Text: Rahim, Karim
- Marks: 85, 90, 75

👉 These values alone are just data, not meaningful information.

📄 Simple Definition

👉 Data = Raw, unprocessed facts

🔴 Key Points

- No context or meaning by itself
- Used as input for processing
- Converted into information after processing

✅ বাংলা উত্তর (Bangla)

ডাটা (Data) হলো **কাঁচা তথ্য বা সংখ্যা**, যার নিজস্ব কোনো অর্থ থাকে না।

- এটি **অপ্রক্রিয়াজাত (unprocessed)** এবং **অসংগঠিত (unorganized)**।
- ডাটা হতে পারে সংখ্যা, লেখা, চিহ্ন বা পর্যবেক্ষণ।
- প্রক্রিয়াজাত না করলে এটি থেকে কোনো অর্থপূর্ণ তথ্য পাওয়া যায় না।

🔴 উদাহরণ:

- সংখ্যা: ১০, ২০, ৩০
- নাম: রহিম, করিম
- নম্বর: ৮৫, ৯০, ৭৫

👉 এগুলো শুধু ডাটা, এগুলোর নিজে কোনো অর্থ নেই।

📄 সহজ সংজ্ঞা

👉 ডাটা = কাঁচা, প্রক্রিয়াবিহীন তথ্য

🔴 মূল পয়েন্ট

- নিজে কোনো অর্থ বহন করে না
- প্রক্রিয়ার ইনপুট হিসেবে ব্যবহৃত হয়
- প্রক্রিয়ার পরে তথ্য (information) তৈরি হয়

2.3 What is field?

♦ What is a Field?

A **field** is a **single attribute or column** in a database table that represents a specific type of data.

- It stores **one piece of information** about an entity.
- Each field has a **name and a data type** (e.g., integer, text, date).
- In a table, fields are represented as **columns**.

🔴 Example:

In a **Student** table:

ID Name Marks

- ID → Field

- Name → Field
- Marks → Field

👉 Each column is a field.

📄 Simple Definition

👉 Field = A column in a database that holds a specific data value

🔴 Key Points

- Represents a single type of data
- Has a name and data type
- Part of a record (row)
- Also called an attribute

✅ বাংলা উত্তর (Bangla)

♦ Field কী?

Field হলো ডাটাবেজ টেবিলের একটি **কলাম (column)**, যা একটি নির্দিষ্ট ধরনের ডাটা ধারণ করে।

- এটি একটি নির্দিষ্ট তথ্য সংরক্ষণ করে
- প্রতিটি field-এর একটি **নাম** এবং **data type** থাকে
- টেবিলে column আকারে থাকে

🔴 উদাহরণ:

Student টেবিলে:

ID Name Marks

- ID → Field
- Name → Field
- Marks → Field

👉 প্রতিটি column-ই একটি field।

📄 সহজ সংজ্ঞা

👉 Field = ডাটাবেজের একটি column যেখানে নির্দিষ্ট তথ্য থাকে

🔴 মূল পয়েন্ট

- একটি নির্দিষ্ট ধরনের ডাটা রাখে
- নাম ও data type থাকে
- row (record)-এর অংশ
- এটাকে attribute-ও বলা হয়

2.4 What is record?

♦ What is a Record?

A **record** is a **collection of related fields** in a database table that represents a single entity.

- It is a **row** in a table.
- A record contains **multiple fields (columns)** with values.

- Each record represents **one complete entry** of data.

✦ Example:

In a **Student** table:

ID	Name	Marks
101	Rahim	85

👉 This entire row is a **record**.

- ID = 101
- Name = Rahim
- Marks = 85

All together form one record.

📄 Simple Definition

👉 Record = A row in a table containing related fields

✦ Key Points

- Also called a **tuple**
- Represents a single entity
- Contains multiple fields
- Stored horizontally in a table

✅ বাংলা উত্তর (Bangla)

◆ Record কী?

Record হলো ডাটাবেজ টেবিলের একটি **row**, যেখানে সম্পর্কিত একাধিক field-এর ডাটা থাকে।

- এটি একটি সম্পূর্ণ তথ্যের সেট
- একটি record-এর মধ্যে একাধিক field থাকে
- প্রতিটি record একটি নির্দিষ্ট entity (যেমন একজন student) কে প্রতিনিধিত্ব করে

✦ উদাহরণ:

Student টেবিলে:

ID	Name	Marks
101	Rahim	85

👉 এই পুরো row-টাই একটি **record**।

- ID = 101
- Name = Rahim
- Marks = 85

সব মিলিয়ে একটি record তৈরি হয়।

📄 সহজ সংজ্ঞা

👉 Record = টেবিলের একটি row যেখানে সম্পর্কিত ডাটা থাকে

✦ মূল পয়েন্ট

- একে tuple-ও বলা হয়
- একটি entity-কে represent করে
- একাধিক field নিয়ে গঠিত
- টেবিলে horizontally থাকে

2.5 What is file?

◆ What is a File?

A **file** is a **collection of related records** stored together in a storage device.

- It is used to store data permanently.
- A file contains multiple records, and each record contains multiple fields.
- Files are part of the traditional file system used before databases.

✦ Example:

A “Student File” may contain many student records:

ID Name Marks

101 Rahim 85

102 Karim 90

👉 This entire collection of records is called a **file**.

📄 Simple Definition

👉 File = A collection of related records stored together

✦ Key Points

- Stores data permanently
- Contains multiple records
- Used in file-based systems (before DBMS)
- Less organized compared to database tables

✅ বাংলা উত্তর (Bangla)

◆ File কী?

File হলো সম্পর্কিত একাধিক **record-এর সমষ্টি**, যা একটি স্টোরেজ ডিভাইসে সংরক্ষিত থাকে।

- এটি ডাটা স্থায়ীভাবে সংরক্ষণ করতে ব্যবহৃত হয়
- একটি file-এর মধ্যে অনেকগুলো record থাকে
- প্রতিটি record আবার অনেকগুলো field নিয়ে গঠিত

✦ উদাহরণ:

একটি “Student File” এ অনেক student-এর তথ্য থাকতে পারে:

ID Name Marks

101 Rahim 85

102 Karim 90

👉 এই সব record মিলেই একটি **file** তৈরি হয়।

📄 সহজ সংজ্ঞা

👉 File = সম্পর্কিত record-এর সমষ্টি

✦ মূল পয়েন্ট

- ডাটা স্থায়ীভাবে সংরক্ষণ করে
- একাধিক record থাকে
- DBMS এর আগে file system ব্যবহৃত হতো
- ডাটাবেজের তুলনায় কম organized

2.6 Make a customer file with all proper content.

♦ Customer File (Example in Database Format)

A **customer file** is a collection of customer records stored in a structured way. Each record contains multiple fields (attributes) related to a customer.

✦ Customer File Structure (Table Format)

Customer_ID	Name	Address	Phone	Email	Gender	Date_of_Birth
C001	Rahim	Rajshahi, BD	01712345678	rahim@gmail.com	Male	1995-05-12
C002	Karim	Dhaka, BD	01823456789	karim@yahoo.com	Male	1993-08-21
C003	Ayesha	Khulna, BD	01934567890	ayesha@gmail.com	Female	1998-02-10
C004	Sumaiya	Chittagong, BD	01645678901	sumaiya@gmail.com	Female	2000-11-05

♦ Fields (Attributes) Explained

- **Customer_ID** → Unique identifier for each customer
- **Name** → Customer name
- **Address** → Location of the customer
- **Phone** → Contact number
- **Email** → Email address
- **Gender** → Male/Female
- **Date_of_Birth** → Birth date

📄 Simple Understanding

👉 Customer File = Collection of all customer records stored in a table format

✅ বাংলা উত্তর (Bangla)

♦ Customer File (উদাহরণসহ)

Customer file হলো এমন একটি ফাইল যেখানে গ্রাহকদের তথ্য টেবিল আকারে সংরক্ষণ করা হয়। প্রতিটি record একটি গ্রাহকের সম্পূর্ণ তথ্য ধারণ করে।

✦ Customer File Structure (Table আকারে)

Customer_ID	Name	Address	Phone	Email	Gender	Date_of_Birth
C001	Rahim	Rajshahi, BD	01712345678	rahim@gmail.com	Male	1995-05-12
C002	Karim	Dhaka, BD	01823456789	karim@yahoo.com	Male	1993-08-21
C003	Ayesha	Khulna, BD	01934567890	ayesha@gmail.com	Female	1998-02-10
C004	Sumaiya	Chittagong, BD	01645678901	sumaiya@gmail.com	Female	2000-11-05

◆ Fields (Attributes) ব্যাখ্যা

- **Customer_ID** → প্রতিটি গ্রাহকের unique ID
- **Name** → গ্রাহকের নাম
- **Address** → ঠিকানা
- **Phone** → ফোন নম্বর
- **Email** → ইমেইল
- **Gender** → লিঙ্গ
- **Date_of_Birth** → জন্ম তারিখ

📁 সহজভাবে

👉 Customer File = গ্রাহকদের সব record একসাথে টেবিল আকারে সংরক্ষণ

3.1 what is entity set?

◆ What is an Entity Set?

An **entity set** is a **collection of similar types of entities** in a database.

- An **entity** is a real-world object (like a student, customer, product).
- An **entity set** groups all entities of the same type together.
- It is usually represented as a **table** in a database.

📌 Example

If we have a **Student entity**, then:

ID Name Marks

101 Rahim 85

102 Karim 90

👉 All these student records together form the **Student Entity Set**.

📁 Simple Definition

👉 Entity Set = A collection of similar entities (same type)

📌 Key Points

- Represents a group of similar objects
- Each row = one entity
- Table = entity set
- Has attributes (fields)

✅ বাংলা উত্তর (Bangla)

◆ Entity Set কী?

Entity set হলো একই ধরনের একাধিক **entity**-এর সমষ্টি।

- **Entity** হলো বাস্তব জীবনের কোনো বস্তু (যেমন student, customer)
- একই ধরনের সব entity একসাথে মিলে entity set তৈরি করে
- এটি সাধারণত ডাটাবেজে একটি **table** হিসেবে থাকে

✦ উদাহরণ

ধরা যাক Student entity:

ID Name Marks

101 Rahim 85

102 Karim 90

👉 এই সব student মিলেই একটি **Student Entity Set**।

📦 সহজ সংজ্ঞা

👉 Entity Set = একই ধরনের entity-এর সমষ্টি

✦ মূল পয়েন্ট

- একই ধরনের object-এর group
- প্রতিটি row = একটি entity
- পুরো table = entity set
- এর attribute (field) থাকে

3.2 What is relationship set?

♦ What is a Relationship Set?

A **relationship set** is a **collection of similar relationships** between two or more entity sets in a database.

- A **relationship** shows how entities are connected.
- A **relationship set** groups all similar relationships together.
- It is used in **ER (Entity-Relationship) model**.

✦ Example

Consider two entity sets:

- **Student**
- **Course**

Relationship: “**Enrolls**”

Student_ID Course_ID

101 CSE101

102 CSE102

👉 These connections form a **relationship set** called **Enrolls**.

📦 Simple Definition

👉 Relationship Set = A collection of similar relationships between entities

✦ Key Points

- Connects two or more entity sets
- Represents association between entities
- Used in ER diagram
- Can have attributes (like date, status)

✅ বাংলা উত্তর (Bangla)

◆ Relationship Set কী?

Relationship set হলো দুই বা ততোধিক entity set-এর মধ্যে সম্পর্কগুলোর সমষ্টি।

- Relationship দেখায় entity গুলোর মধ্যে কী সম্পর্ক আছে
- একই ধরনের সব relationship একসাথে মিলে relationship set তৈরি করে
- এটি ER diagram-এ ব্যবহৃত হয়

✦ উদাহরণ

ধরা যাক:

- Student
- Course

Relationship: Enrolls (ভর্তি হওয়া)

Student_ID Course_ID

101 CSE101

102 CSE102

👉 এই সম্পর্কগুলো মিলেই একটি relationship set (Enrolls) তৈরি করে।

📁 সহজ সংজ্ঞা

👉 Relationship Set = entity গুলোর মধ্যে সম্পর্কের সমষ্টি

✦ মূল পয়েন্ট

- একাধিক entity set কে সংযুক্ত করে
- তাদের মধ্যে সম্পর্ক প্রকাশ করে
- ER diagram-এ ব্যবহৃত হয়
- এর নিজস্ব attribute থাকতে পারে

3.3 What is recursive relation?

◆ What is a Recursive Relationship?

A recursive relationship (also called a unary relationship) is a relationship where an entity is related to itself.

- It occurs within the same entity set.
- One entity instance is connected to another instance of the same type.

✦ Example

Consider an entity set: Employee

Relationship: “**Manages**”

Employee_ID Manager_ID

101	201
102	201

👉 Here:

- Employee 201 manages Employee 101 and 102
- Both are from the same entity set (**Employee**)

✓ This is a **recursive relationship**.

📄 Simple Definition

👉 Recursive Relation = A relationship where an entity relates to itself

🔴 Key Points

- Involves only **one entity set**
- Also called **Unary Relationship**
- Common in hierarchical structures
- Example: Employee–Manager, Teacher–Mentor

✅ বাংলা উত্তর (Bangla)

◆ Recursive Relation কী?

Recursive relationship (বা **Unary relationship**) হলো এমন একটি সম্পর্ক যেখানে একটি **entity** নিজেই নিজের সাথে সম্পর্কিত হয়।

- এটি একই entity set-এর মধ্যে ঘটে
- একটি entity অন্য একই ধরনের entity-এর সাথে সম্পর্ক তৈরি করে

🔴 উদাহরণ

ধরা যাক entity set: **Employee**

Relationship: **Manages** (পরিচালনা করে)

Employee_ID Manager_ID

101	201
102	201

👉 এখানে:

- Employee 201, Employee 101 এবং 102-কে manage করে
- সবাই একই entity set (**Employee**) এর অংশ

✓ এটি একটি **recursive relationship**।

📄 সহজ সংজ্ঞা

👉 Recursive Relation = একটি entity নিজেই নিজের সাথে সম্পর্ক তৈরি করে

🔴 মূল পয়েন্ট

- একটিমাত্র entity set থাকে
- একে Unary relationship-ও বলা হয়
- hierarchy (উর্ধ্ব-নিম্ন সম্পর্ক) বোঝাতে ব্যবহৃত হয়

- উদাহরণ: Employee–Manager

3.4 What is er diagram? The content of er diagram.

♦ What is an ER Diagram?

An **ER Diagram (Entity-Relationship Diagram)** is a **graphical representation** of a database that shows:

- **Entities**
- **Attributes**
- **Relationships**

👉 It is used during **database design** to visually model how data is organized and related.

♦ Contents (Components) of ER Diagram

An ER diagram mainly consists of the following elements:

1. Entity

- A real-world object (e.g., Student, Customer)
- Represented by a **rectangle**

✚ Example: Student, Employee

2. Attribute

- Describes properties of an entity
- Represented by an **oval (ellipse)**

✚ Example: Name, ID, Age

3. Relationship

- Shows how entities are related
- Represented by a **diamond shape**

✚ Example: Student “enrolls” in Course

4. Primary Key

- A unique attribute that identifies each entity
- Usually **underlined**

✚ Example: Student_ID

5. Cardinality (Mapping)

- Defines the number of relationships between entities
- Types:
 - One-to-One (1:1)
 - One-to-Many (1:M)
 - Many-to-Many (M:N)

6. Weak Entity

- Entity that depends on another entity
- Represented by a **double rectangle**

7. Participation Constraint

- Shows whether participation is **total or partial**

✦ Simple Example (Concept)

- Entity: Student
- Entity: Course
- Relationship: Enrolls

👉 Student enrolls in Course

📄 Simple Definition

👉 ER Diagram = Visual design of database structure

✅ বাংলা উত্তর (Bangla)

◆ ER Diagram কী?

ER Diagram (Entity-Relationship Diagram) হলো একটি চিত্র (diagram) যা ডাটাবেজের গঠন এবং সম্পর্কগুলোকে দেখায়।

👉 এটি ডাটাবেজ ডিজাইন করার সময় ব্যবহার করা হয়।

◆ ER Diagram-এর উপাদান (Contents)

১. Entity (এনটিটি)

- বাস্তব জীবনের কোনো বস্তু
- Rectangle (বক্স) দ্বারা দেখানো হয়

✦ উদাহরণ: Student, Customer

২. Attribute (অ্যাট্রিবিউট)

- Entity-এর বৈশিষ্ট্য
- Oval (ডিম্বাকৃতি) দ্বারা দেখানো হয়

✦ উদাহরণ: Name, ID, Age

৩. Relationship (সম্পর্ক)

- Entity গুলোর মধ্যে সম্পর্ক
- Diamond (হীরার মতো আকৃতি) দ্বারা দেখানো হয়

✦ উদাহরণ: Student → Enrolls → Course

৪. Primary Key

- একটি unique attribute যা entity কে আলাদা করে
- সাধারণত underline করা থাকে

✦ উদাহরণ: Student_ID

৫. Cardinality (Mapping)

- কত সংখ্যক entity একে অপরের সাথে সম্পর্কিত
- ধরন:

- 1:1
- 1:M
- M:N

৬. Weak Entity

- অন্য entity-এর উপর নির্ভরশীল entity
- Double rectangle দ্বারা দেখানো হয়

৭. Participation Constraint

- Entity সম্পর্কের মধ্যে পুরোপুরি যুক্ত কিনা তা বোঝায়

✦ সহজ উদাহরণ

- Student
- Course
- Enrolls

👉 Student course-এ ভর্তি হয়

📄 সহজ সংজ্ঞা

👉 ER Diagram = ডাটাবেজের চিত্র আকারে উপস্থাপন

3.5 Describe about mapping cardinalities? In details.

◆ What is Mapping Cardinality?

Mapping cardinality (or **cardinality ratio**) defines the **number of entities of one entity set that can be associated with the number of entities of another entity set** in a relationship.

👉 It tells us **how many instances of one entity relate to another.**

◆ Types of Mapping Cardinalities

There are mainly **three types**:

◆ 1. One-to-One (1:1) Relationship

- One entity from **Entity A** is related to **only one entity** of Entity B
- And vice versa

✦ Example:

- One person has **one passport**
- One passport belongs to **one person**

👉 Person ↔ Passport

✓ No duplication on either side

◆ 2. One-to-Many (1:M) Relationship

- One entity from **Entity A** can be related to **many entities** of Entity B
- But each entity in B is related to **only one** in A

✦ Example:

- One teacher teaches **many students**

- But each student has **one teacher**

👉 Teacher → Students

✓ Most common relationship

♦ 3. Many-to-Many (M:N) Relationship

- One entity from A can relate to **many entities** in B
- And one entity from B can relate to **many entities** in A

✦ **Example:**

- Students enroll in **many courses**
- Courses have **many students**

👉 Student ↔ Course

✓ Requires a **junction/bridge table** in relational database

♦ Visual Idea (Simple)

- 1:1 → A ↔ B
- 1:M → A → B B B
- M:N → A A ↔ B B

♦ Importance of Mapping Cardinality

- Helps in **database design**
- Determines how tables are **linked**
- Avoids **data redundancy**
- Ensures proper **relationship modeling**

📄 Simple Summary

👉 Mapping Cardinality = Defines **how many entities relate to each other**

✅ বাংলা উত্তর (Bangla)

♦ Mapping Cardinality কী?

Mapping cardinality হলো এমন একটি ধারণা যা দেখায় একটি entity-এর সাথে অন্য entity-এর **কতগুলো instance সম্পর্কিত হতে পারে।**

👉 সহজভাবে: এক entity কতগুলোর সাথে সম্পর্ক করতে পারে

♦ Mapping Cardinality-এর প্রকারভেদ

♦ ১. One-to-One (1:1)

- একটি entity → অন্য entity-এর **একটির সাথে** সম্পর্কিত
- উল্টোটাও একই

✦ **উদাহরণ:**

- একজন মানুষ → একটি passport
- একটি passport → একজন মানুষ

👉 Person ↔ Passport

♦ ২. One-to-Many (1:M)

- একটি entity → অনেকগুলোর সাথে সম্পর্কিত

- কিন্তু প্রতিটি entity → একটির সাথে সম্পর্কিত

✦ উদাহরণ:

- একজন শিক্ষক → অনেক ছাত্র
- একজন ছাত্র → একজন শিক্ষক

👉 Teacher → Students

♦ ৩. Many-to-Many (M:N)

- একটি entity → অনেকগুলোর সাথে সম্পর্কিত
- এবং অন্যটাও → অনেকগুলোর সাথে সম্পর্কিত

✦ উদাহরণ:

- ছাত্র → অনেক course
- course → অনেক ছাত্র

👉 Student ↔ Course

✓ এখানে extra table লাগে (junction table)

♦ গুরুত্ব (Importance)

- ডাটাবেজ ডিজাইন সহজ করে
- টেবিলের সম্পর্ক ঠিকভাবে তৈরি করতে সাহায্য করে
- ডাটা duplication কমায়
- সঠিক structure নিশ্চিত করে

📦 সহজভাবে

👉 Mapping Cardinality = entity গুলোর মধ্যে সম্পর্কের সংখ্যা নির্ধারণ করে

3.6 What is super key, primary key and candidate key and foreign key?

♦ 1. Super Key

A **super key** is a **set of one or more attributes** that can uniquely identify a record in a table.

- It may contain **extra attributes** (not minimal).
- Any combination that uniquely identifies a row is a super key.

✦ Example:

In a Student table (ID, Name, Email):

- {ID}
- {ID, Name}
- {ID, Email}

👉 All are **super keys** because they uniquely identify a record.

♦ 2. Candidate Key

A **candidate key** is a **minimal super key**.

- It uniquely identifies a record
- It has **no unnecessary attributes**

✦ Example:

- {ID}
- {Email}

👉 Both can uniquely identify a student and are minimal → **candidate keys**

◆ 3. Primary Key

A **primary key** is **one selected candidate key** used to uniquely identify records.

- Cannot be **NULL**
- Must be **unique**
- Only **one primary key** per table

✦ Example:

- ID is chosen as the primary key
-

◆ 4. Foreign Key

A **foreign key** is an attribute in one table that **refers to the primary key of another table**.

- It creates a **relationship between tables**
- Can have duplicate values

✦ Example:

Student Table

ID Name

Course Table

Course_ID Student_ID

👉 Student_ID in Course table is a **foreign key**

📋 Summary Table

Key Type	Description
Super Key	Any attribute(s) that uniquely identify
Candidate Key	Minimal super key
Primary Key	Selected candidate key
Foreign Key	Refers to primary key of another table

✦ Simple Understanding

👉 Super Key $\supseteq$ Candidate Key $\rightarrow$ Primary Key

👉 Foreign Key $\rightarrow$ Connects tables

✅ বাংলা উত্তর (Bangla)

◆ ১. Super Key

Super key হলো এমন attribute বা attribute-এর সমষ্টি যা একটি record কে uniquely identify করতে পারে।

- এতে অতিরিক্ত attribute থাকতে পারে

✦ উদাহরণ:

Student table:

- {ID}
- {ID, Name}
- {ID, Email}

👉 সবগুলোই super key

◆ ২. Candidate Key

Candidate key হলো **minimal super key**।

- কোনো অতিরিক্ত attribute থাকে না
- uniquely identify করে

✦ **উদাহরণ:**

- {ID}
- {Email}

👉 এগুলো candidate key

◆ ৩. Primary Key

Primary key হলো candidate key থেকে একটি নির্বাচন করা key।

- Unique হতে হবে
- NULL হতে পারবে না
- একটি table-এ একটি primary key থাকে

✦ **উদাহরণ:**

- ID → Primary key

◆ ৪. Foreign Key

Foreign key হলো একটি table-এর attribute যা অন্য table-এর **primary key**-কে refer করে।

- table-এর মধ্যে relationship তৈরি করে

✦ **উদাহরণ:**

Student Table:

| ID | Name |

Course Table:

| Course_ID | Student_ID |

👉 Student_ID → Foreign key

📄 সংক্ষেপে

Key Type	বর্ণনা
Super Key	যেকোনো unique attribute set
Candidate Key	minimal super key
Primary Key	নির্বাচিত candidate key
Foreign Key	অন্য table-এর primary key refer করে

✦ **সহজভাবে**

👉 Super $\supseteq$ Candidate $\rightarrow$ Primary

👉 Foreign Key $\rightarrow$ table গুলোকে যুক্ত করে

3.7 What is Binary relation? With example.

◆ What is a Binary Relationship?

A **binary relationship** is a type of relationship in a database where **two entity sets are involved**.

👉 It shows how **two different entities are related to each other**.

- “Bi” means **two**
- So, binary relationship = relationship between **two entities**

◆ Example of Binary Relationship

Consider two entity sets:

- **Student**
- **Course**

Relationship: **Enrolls**

📌 Representation:

Student_ID Course_ID

101 CSE101

102 CSE102

👉 Here:

- A student enrolls in a course
- Two entities (Student & Course) are involved

✓ This is a **binary relationship**

◆ More Examples

- Employee **works in** Department
- Customer **buys** Product
- Teacher **teaches** Student

◆ Key Points

- Involves **exactly two entity sets**
- Most common type of relationship
- Can be 1:1, 1:M, or M:N

📄 Simple Definition

👉 Binary Relation = Relationship between two entity sets

✅ বাংলা উত্তর (Bangla)

◆ Binary Relation কী?

Binary relation হলো এমন একটি relationship যেখানে **দুইটি entity set জড়িত থাকে**।

👉 অর্থাৎ, দুইটি entity-এর মধ্যে সম্পর্ক

- “Bi” মানে **দুই**
- তাই binary relationship = দুই entity-এর মধ্যে সম্পর্ক

◆ উদাহরণ

ধরা যাক:

- **Student**

- **Course**

Relationship: **Enrolls**

🔴 টেবিল:

Student_ID Course_ID

101 CSE101

102 CSE102

👉 এখানে:

- Student course-এ ভর্তি হয়
- দুইটি entity আছে (Student & Course)

✓ এটি একটি **binary relationship**

♦ **আরও উদাহরণ**

- Employee → Department (কাজ করে)
- Customer → Product (কিনে)
- Teacher → Student (পড়ায়)

♦ **মূল পয়েন্ট**

- দুইটি entity set থাকে
- সবচেয়ে বেশি ব্যবহৃত relationship
- 1:1, 1:M, M:N হতে পারে

🔵 **সহজ সংজ্ঞা**

👉 Binary Relation = দুই entity-এর মধ্যে সম্পর্ক

3.8 What is ternary relation? With example.

♦ **What is a Ternary Relationship?**

A **ternary relationship** is a relationship in a database where **three entity sets are involved at the same time**.

👉 “Ternary” means **three**, so it represents a relationship among **three different entities**.

♦ **Example of Ternary Relationship**

Consider three entity sets:

- **Supplier**
- **Product**
- **Project**

Relationship: **Supplies**

🔴 Representation:

Supplier_ID Product_ID Project_ID

S01 P01 PR01

S02 P02 PR02

👉 Meaning:

- A supplier supplies a specific product to a specific project

✓ This relationship cannot be properly represented using only binary relationships.

◆ Another Example

- **Doctor, Patient, Medicine**
- Relationship: **Prescribes**

👉 A doctor prescribes a medicine to a patient

◆ Key Points

- Involves **three entity sets**
- More complex than binary relationship
- All three entities are interdependent
- Represented by a **diamond connected to three entities** in ER diagram

📄 Simple Definition

👉 Ternary Relation = Relationship among three entity sets

✅ বাংলা উত্তর (Bangla)

◆ Ternary Relation কী?

Ternary relation হলো এমন একটি relationship যেখানে **তিনটি entity set একসাথে যুক্ত থাকে।**

👉 “Ternary” মানে **তিন**, অর্থাৎ তিনটি entity-এর মধ্যে সম্পর্ক

◆ উদাহরণ

ধরা যাক:

- **Supplier**
- **Product**
- **Project**

Relationship: **Supplies**

✦ টেবিল:

Supplier_ID Product_ID Project_ID

S01 P01 PR01

S02 P02 PR02

👉 অর্থ:

- একটি supplier একটি product নির্দিষ্ট project-এ সরবরাহ করে

✓ এটি binary দিয়ে ঠিকভাবে বোঝানো যায় না

◆ আরও উদাহরণ

- **Doctor, Patient, Medicine**
- Relationship: **Prescribes**

👉 Doctor patient-কে medicine দেয়

◆ মূল পয়েন্ট

- তিনটি entity set থাকে
- binary relation-এর চেয়ে জটিল

- তিনটিই একে অপরের উপর নির্ভরশীল
- ER diagram-এ তিনটি entity-এর সাথে diamond যুক্ত থাকে

📖 সহজ সংজ্ঞা

👉 Ternary Relation = তিনটি entity-এর মধ্যে সম্পর্ক

3.9 Describe the process of replacing ternary relation.

♦ What is Replacing a Ternary Relationship?

Replacing a **ternary relationship** means converting a relationship involving **three entity sets** into **multiple binary relationships**.

👉 This is often done when designing a **relational database schema**, because relational databases handle binary relations more easily.

♦ ⚠ Important Idea

A ternary relationship **cannot always be perfectly replaced** by simple binary relationships without losing meaning.

👉 So, we use a **new entity (associative entity / junction table)**.

♦ Process of Replacing Ternary Relation

Step 1: Identify the Ternary Relationship

Example:

- Entities: **Supplier, Product, Project**
- Relationship: **Supplies**

Step 2: Create a New Entity (Relation/Table)

- Create a new table named after the relationship (e.g., **Supplies**)

Step 3: Add Foreign Keys

- Include primary keys of all three entities as **foreign keys** in the new table

📌 Example:

Supplies Table

Supplier_ID Product_ID Project_ID

Step 4: Define Composite Primary Key

- Combine all foreign keys to form a **composite primary key**

👉 (Supplier_ID + Product_ID + Project_ID)

Step 5: Add Attributes (if any)

- If the relationship has attributes (e.g., Quantity), include them

📌 Example:

Supplier_ID Product_ID Project_ID Quantity

♦ Alternative (Not Recommended Alone)

Breaking into 3 binary relations like:

- Supplier → Product
- Product → Project
- Supplier → Project

✗ This **loses the real meaning** (who supplies what to which project)

◆ Why This Method is Used

- Maintains correct relationship meaning
- Avoids data loss
- Fits relational model properly

📄 Simple Summary

👉 Ternary relation → Create new table → Add 3 foreign keys → Use composite key

✅ বাংলা উত্তর (Bangla)

◆ Ternary Relation Replace করা কী?

Ternary relation replace করা মানে হলো **তিনটি entity-এর relationship-কে relational database-এ রূপান্তর করা।**

👉 সাধারণত এটি একটি **নতুন table (associative entity)** তৈরি করে করা হয়।

◆ ⚠️ গুরুত্বপূর্ণ বিষয়

শুধু binary relation-এ ভেঙে ফেললে অনেক সময় **মূল অর্থ হারিয়ে যায়।**

👉 তাই নতুন একটি table তৈরি করা হয়

◆ ধাপে ধাপে প্রক্রিয়া

ধাপ ১: Ternary Relationship চিহ্নিত করা

উদাহরণ:

- Supplier, Product, Project
- Relationship: Supplies

ধাপ ২: নতুন Table তৈরি করা

- Relationship অনুযায়ী একটি নতুন table তৈরি করা

👉 যেমন: Supplies

ধাপ ৩: Foreign Key যোগ করা

- তিনটি entity-এর primary key → নতুন table-এ foreign key হিসেবে যোগ করা

📌 Example:

| Supplier_ID | Product_ID | Project_ID |

ধাপ ৪: Composite Primary Key তৈরি করা

- তিনটি key একসাথে primary key হিসেবে ব্যবহার করা

👉 (Supplier_ID + Product_ID + Project_ID)

ধাপ ৫: Attribute যোগ করা (যদি থাকে)

- যেমন Quantity, Date ইত্যাদি

🔴 Example:

| Supplier_ID | Product_ID | Project_ID | Quantity |

♦ ভুল পদ্ধতি (Avoid করা উচিত)

- আলাদা আলাদা binary relation বানানো

✗ এতে আসল সম্পর্ক বোঝা যায় না

♦ কেন এই পদ্ধতি ব্যবহার করা হয়

- সঠিক সম্পর্ক বজায় রাখে
- ডাটা loss হয় না
- relational model-এর সাথে মানানসই

📦 সহজভাবে

👉 Ternary relation → নতুন table → ৩টি foreign key → composite key

4.1 What is DDL? possible all example based on DBMS MS SQL.

♦ What is DDL?

DDL (Data Definition Language) is a set of SQL commands used to **define and manage the structure of a database**.

👉 It is used to **create, modify, and delete database objects** like tables, schemas, indexes, etc.

♦ Main DDL Commands in MS SQL (with Examples)

♦ 1. CREATE

Used to **create database objects** like tables.

🔴 Example (Create Table):

```
CREATE TABLE Student (  
ID INT PRIMARY KEY,  
Name VARCHAR(50),  
Age INT,  
Department VARCHAR(50)  
);
```

♦ 2. ALTER

Used to **modify an existing table**.

🔴 Example (Add Column):

```
ALTER TABLE Student  
ADD Email VARCHAR(100);
```

🔴 Example (Modify Column):

```
ALTER TABLE Student  
ALTER COLUMN Name VARCHAR(100);
```

◆ 3. DROP

Used to **delete a table or database completely**.

✦ Example:

```
DROP TABLE Student;
```

👉 This removes the table permanently.

◆ 4. TRUNCATE

Used to **delete all records from a table** but keeps the structure.

✦ Example:

```
TRUNCATE TABLE Student;
```

👉 Faster than DELETE and cannot be rolled back in most cases.

◆ 5. RENAME (In MS SQL → using sp_rename)

Used to **rename a table or column**.

✦ Example:

```
EXEC sp_rename 'Student', 'Student_Info';
```

◆ Key Features of DDL

- Works on **structure, not data**
- Automatically **commits changes**
- Cannot be easily rolled back

📄 Simple Summary

👉 DDL = Defines database structure (Create, Modify, Delete)

✅ বাংলা উত্তর (Bangla)

◆ DDL কী?

DDL (Data Definition Language) হলো SQL-এর এমন কিছু command, যা ব্যবহার করে **ডাটাবেজের structure তৈরি ও পরিবর্তন করা হয়**।

👉 এটি table, database ইত্যাদি তৈরি, পরিবর্তন ও মুছে ফেলার জন্য ব্যবহৃত হয়।

◆ MS SQL-এ DDL Command গুলো

◆ ১. CREATE

নতুন table বা database তৈরি করতে ব্যবহৃত হয়।

✦ উদাহরণ:

```
CREATE TABLE Student (  
ID INT PRIMARY KEY,  
Name VARCHAR(50),  
Age INT,  
Department VARCHAR(50)  
);
```

◆ ২. ALTER

বিদ্যমান table পরিবর্তন করতে ব্যবহৃত হয়।

✦ Column যোগ:

```
ALTER TABLE Student  
ADD Email VARCHAR(100);
```

✦ Column পরিবর্তন:

```
ALTER TABLE Student  
ALTER COLUMN Name VARCHAR(100);
```

◆ ৩. DROP

সম্পূর্ণ table বা database মুছে ফেলতে ব্যবহৃত হয়।

✦ উদাহরণ:

```
DROP TABLE Student;
```

◆ ৪. TRUNCATE

Table-এর সব data মুছে ফেলে কিন্তু structure রেখে দেয়।

✦ উদাহরণ:

```
TRUNCATE TABLE Student;
```

◆ ৫. RENAME (sp_rename)

Table বা column-এর নাম পরিবর্তন করতে ব্যবহৃত হয়।

✦ উদাহরণ:

```
EXEC sp_rename 'Student', 'Student_Info';
```

◆ মূল বৈশিষ্ট্য

- Structure নিয়ে কাজ করে
- Auto commit হয়
- সহজে rollback করা যায় না

📁 সহজভাবে

👉 DDL = ডাটাবেজের structure তৈরি ও পরিবর্তনের command

4.2 What is DML? possible all example based on DBMS MS SQL.

◆ What is DML?

DML (Data Manipulation Language) is a set of SQL commands used to **manipulate the data stored in database tables**.

- Unlike DDL, DML deals with **data inside the tables**, not the structure.
- Common DML operations include: **Insert, Update, Delete, Select**.

◆ Main DML Commands in MS SQL (with Examples)

◆ 1. INSERT

Used to **add new records** into a table.

✦ Example:

```
INSERT INTO Student (ID, Name, Age, Department)
VALUES (101, 'Rahim', 22, 'CSE');
```

✦ Insert multiple rows:

```
INSERT INTO Student (ID, Name, Age, Department)
VALUES
(102, 'Karim', 23, 'EEE'),
(103, 'Ayesha', 21, 'BBA');
```

◆ 2. UPDATE

Used to **modify existing records** in a table.

✦ Example:

```
UPDATE Student
SET Age = 23
WHERE ID = 101;
```

✦ Update multiple columns:

```
UPDATE Student
SET Age = 24, Department = 'EEE'
WHERE ID = 102;
```

◆ 3. DELETE

Used to **remove records** from a table.

✦ Example:

```
DELETE FROM Student
WHERE ID = 103;
```

✦ Delete all records:

```
DELETE FROM Student;
```

⚠ Unlike TRUNCATE, DELETE can be rolled back if wrapped in a transaction.

◆ 4. SELECT

Used to **retrieve data** from a table.

✦ Example (all records):

```
SELECT * FROM Student;
```

✦ Example (specific columns):

```
SELECT Name, Department FROM Student;
```

✦ Example with condition:

```
SELECT * FROM Student
WHERE Age > 22;
```

◆ Key Features of DML

- Works on **data, not structure**
- Can **insert, update, delete, or retrieve** data
- Supports conditions and filters
- Changes can be **rolled back** if inside a transaction

📄 Simple Summary

👉 DML = Manipulates data inside tables (Insert, Update, Delete, Select)

✅ বাংলা উত্তর (Bangla)

◆ DML কী?

DML (Data Manipulation Language) হলো SQL-এর এমন command, যা ডাটাবেজের table-এর ভিতরের data পরিচালনা করে।

- DDL এর মত structure নয়, এখানে data নিয়ন্ত্রণ হয়
- মূল DML operations: **Insert, Update, Delete, Select**

◆ MS SQL-এ DML Commands

◆ ১. INSERT

নতুন record যোগ করতে ব্যবহৃত হয়।

✦ উদাহরণ:

```
INSERT INTO Student (ID, Name, Age, Department)
VALUES (101, 'Rahim', 22, 'CSE');
```

✦ একাধিক row insert:

```
INSERT INTO Student (ID, Name, Age, Department)
VALUES
(102, 'Karim', 23, 'EEE'),
(103, 'Ayesha', 21, 'BBA');
```

◆ ২. UPDATE

বিদ্যমান record পরিবর্তন করতে ব্যবহৃত হয়।

✦ উদাহরণ:

```
UPDATE Student
SET Age = 23
WHERE ID = 101;
```

✦ একাধিক column update:

```
UPDATE Student
SET Age = 24, Department = 'EEE'
WHERE ID = 102;
```

◆ ৩. DELETE

Record মুছে ফেলতে ব্যবহৃত হয়।

✦ উদাহরণ:

```
DELETE FROM Student
WHERE ID = 103;
```

✦ সব record মুছে ফেলা:

```
DELETE FROM Student;
```

⚠ DELETE rollback করা যায়, TRUNCATE নয়

◆ ৪. SELECT

Table থেকে data retrieve করতে ব্যবহৃত হয়।

✦ সব record:

```
SELECT * FROM Student;
```

✦ নির্দিষ্ট column:

```
SELECT Name, Department FROM Student;
```

✦ শর্ত দিয়ে:

```
SELECT * FROM Student
```

```
WHERE Age > 22;
```

◆ মূল বৈশিষ্ট্য

- Data নিয়ন্ত্রণ করে, structure নয়
- Insert, Update, Delete, Select করা যায়
- Condition/Filter ব্যবহার করা যায়
- Transaction এ rollback সম্ভব

📦 সহজ সংজ্ঞা

👉 DML = Table-এর data manipulate করা (Insert, Update, Delete, Select)

4.3 Create a database for an university and differentiate the DDL and DML using MS SQL.

◆ Create a University Database (MS SQL Example)

✦ Step 1: Create Database (DDL)

```
CREATE DATABASE UniversityDB;
```

✦ Step 2: Use the Database

```
USE UniversityDB;
```

◆ Create Tables (DDL)

✦ Student Table

```
CREATE TABLE Student (  
StudentID INT PRIMARY KEY,  
Name VARCHAR(50),  
Age INT,  
Department VARCHAR(50)  
);
```

✦ Course Table

```
CREATE TABLE Course (  
CourseID INT PRIMARY KEY,  
CourseName VARCHAR(100),  
Credits INT  
);
```

✦ Enrollment Table (Relationship)

```
CREATE TABLE Enrollment (  
StudentID INT,  
CourseID INT,  
EnrollmentDate DATE,  
PRIMARY KEY (StudentID, CourseID),
```

```
FOREIGN KEY (StudentID) REFERENCES Student(StudentID),  
FOREIGN KEY (CourseID) REFERENCES Course(CourseID)  
);
```

◆ Insert Data (DML)

```
INSERT INTO Student VALUES (1, 'Rahim', 22, 'CSE');  
INSERT INTO Student VALUES (2, 'Karim', 23, 'EEE');
```

```
INSERT INTO Course VALUES (101, 'Database Systems', 3);  
INSERT INTO Course VALUES (102, 'Computer Networks', 3);
```

```
INSERT INTO Enrollment VALUES (1, 101, '2026-01-01');  
INSERT INTO Enrollment VALUES (2, 102, '2026-01-02');
```

◆ Retrieve Data (DML)

```
SELECT * FROM Student;  
SELECT * FROM Course;  
SELECT * FROM Enrollment;
```

◆ Update Data (DML)

```
UPDATE Student  
SET Age = 24  
WHERE StudentID = 1;
```

◆ Delete Data (DML)

```
DELETE FROM Student  
WHERE StudentID = 2;
```

🔥 Difference Between DDL and DML

Feature	DDL (Data Definition Language)	DML (Data Manipulation Language)
Purpose	Defines database structure	Manages data inside tables
Works on	Schema (tables, database)	Records (rows)
Commands	CREATE, ALTER, DROP, TRUNCATE	INSERT, UPDATE, DELETE, SELECT
Effect	Changes structure	Changes data
Transaction	Auto-commit	Can be rolled back
Example	CREATE TABLE Student	INSERT INTO Student VALUES(...)

📄 Simple Summary

- **DDL** → Database structure তৈরি ও পরিবর্তন করে
- **DML** → Table-এর ভিতরের data insert, update, delete, select করে

✅ বাংলা উত্তর (Bangla)

◆ University Database তৈরি (MS SQL)

🔴 Step 1: Database তৈরি (DDL)

```
CREATE DATABASE UniversityDB;
```

🚀 Step 2: Database ব্যবহার

USE UniversityDB;

◆ Table তৈরি (DDL)

🚀 Student Table

```
CREATE TABLE Student (  
  StudentID INT PRIMARY KEY,  
  Name VARCHAR(50),  
  Age INT,  
  Department VARCHAR(50)  
);
```

🚀 Course Table

```
CREATE TABLE Course (  
  CourseID INT PRIMARY KEY,  
  CourseName VARCHAR(100),  
  Credits INT  
);
```

🚀 Enrollment Table

```
CREATE TABLE Enrollment (  
  StudentID INT,  
  CourseID INT,  
  EnrollmentDate DATE,  
  PRIMARY KEY (StudentID, CourseID),  
  FOREIGN KEY (StudentID) REFERENCES Student(StudentID),  
  FOREIGN KEY (CourseID) REFERENCES Course(CourseID)  
);
```

◆ Data Insert (DML)

```
INSERT INTO Student VALUES (1, 'Rahim', 22, 'CSE');  
INSERT INTO Student VALUES (2, 'Karim', 23, 'EEE');
```

```
INSERT INTO Course VALUES (101, 'Database Systems', 3);  
INSERT INTO Course VALUES (102, 'Computer Networks', 3);
```

```
INSERT INTO Enrollment VALUES (1, 101, '2026-01-01');  
INSERT INTO Enrollment VALUES (2, 102, '2026-01-02');
```

◆ Data Retrieve (DML)

```
SELECT * FROM Student;  
SELECT * FROM Course;  
SELECT * FROM Enrollment;
```

◆ Update Data (DML)

```
UPDATE Student  
SET Age = 24  
WHERE StudentID = 1;
```

◆ Delete Data (DML)


```
DELETE FROM Student
WHERE StudentID = 2;
```

🔥 DDL vs DML পার্থক্য

বিষয়	DDL	DML
কাজ	Structure তৈরি/পরিবর্তন	Data পরিচালনা
কাজের ক্ষেত্র	Table, Database	Table-এর record
Command	CREATE, ALTER, DROP	INSERT, UPDATE, DELETE, SELECT
প্রভাব	Schema পরিবর্তন	Data পরিবর্তন
Transaction	Auto commit	Rollback সম্ভব
উদাহরণ	CREATE TABLE	INSERT INTO

📄 সহজভাবে

- **DDL** → database structure তৈরি করে
- **DML** → সেই structure-এর ভিতরের data নিয়ে কাজ করে

5.0 Database management by the programming language.

5.1 how to create a database from story problem set including creating ER diagram.

◆ Step 1: Identify Entities from the Story

From the problem, we extract the main entities:

- **Bus**
- **Route**
- **Town**
- **Driver**
- **Garage**
- **Stage (Journey segment of a route)**

◆ Step 2: Identify Attributes

◆ Bus

- RegistrationNo (Primary Key)
- Capacity
- Type (Single/Double-deck)

◆ Route

- RouteNo (Primary Key)
- AvgPassengersPerDay

◆ Town

- TownID (Primary Key)
- TownName

◆ Driver

- EmpNo (Primary Key)
- Name
- Address
- Phone (optional)

◆ Garage

- GarageID (Primary Key)
- Location

◆ Stage

- StageID (Primary Key)
- (Represents a segment of a route)

◆ Step 3: Identify Relationships

- A **Route** has many **Buses** (1:M)
- A **Bus** is assigned to one **Route** (M:1)
- A **Route** passes through many **Towns** (M:N)
- A **Stage** belongs to a **Route**
- A **Driver** is assigned to a **Stage** (M:N)
- A **Town** may have a **Garage** (1:1 or optional)
- A **Bus** is stored in a **Garage** (M:1)

◆ Step 4: ER Diagram (Text Representation)

Bus (RegistrationNo, Capacity, Type)

|

| M:1

|

Route (RouteNo, AvgPassengers)

Route ---- passes through ---- Town (TownID, TownName)

Route ---- has ---- Stage (StageID)

Stage ---- assigned to ---- Driver (EmpNo, Name, Address, Phone)

Town ---- has ---- Garage (GarageID, Location)

Bus ---- stored in ---- Garage

◆ Step 5: Convert to Tables (Relational Schema)



Bus

Bus(RegistrationNo PK, Capacity, Type, RouteNo FK, GarageID FK)



Route

Route(RouteNo PK, AvgPassengersPerDay)



Town

Town(TownID PK, TownName)

✦ **Route_Town (M:N)**

Route_Town(RouteNo FK, TownID FK)

PRIMARY KEY (RouteNo, TownID)

✦ **Driver**

Driver(EmpNo PK, Name, Address, Phone)

✦ **Stage**

Stage(StageID PK, RouteNo FK)

✦ **Driver_Stage (M:N)**

Driver_Stage(EmpNo FK, StageID FK)

PRIMARY KEY (EmpNo, StageID)

✦ **Garage**

Garage(GarageID PK, Location, TownID FK)

◆ **Step 6: Summary**

- Identify entities → extract nouns
- Identify relationships → verbs
- Define attributes → properties
- Draw ER diagram → visualize relationships
- Convert ER → relational tables

✅ **বাংলা উত্তর (Bangla)**

◆ **Step 1: Entity চিহ্নিত করা**

Story থেকে main entity গুলো:

- Bus
- Route
- Town
- Driver
- Garage
- Stage

◆ **Step 2: Attributes নির্ধারণ**

◆ **Bus**

- RegistrationNo (PK)
- Capacity
- Type

◆ **Route**

- RouteNo (PK)
- AvgPassengersPerDay

◆ **Town**

- TownID (PK)
- TownName

◆ **Driver**

- EmpNo (PK)
- Name
- Address
- Phone (optional)

♦ **Garage**

- GarageID (PK)
- Location

♦ **Stage**

- StageID (PK)

♦ **Step 3: Relationships**

- Route → Bus (1:M)
- Route ↔ Town (M:N)
- Driver ↔ Stage (M:N)
- Town → Garage (1:1/optional)
- Bus → Garage (M:1)

♦ **Step 4: ER Diagram (Conceptual)**

Bus ---- belongs to ---- Route

Route ---- passes through ---- Town

Route ---- has ---- Stage

Stage ---- assigned to ---- Driver

Town ---- has ---- Garage

Bus ---- stored in ---- Garage

♦ **Step 5: Table Conversion**

Bus

Bus(RegistrationNo PK, Capacity, Type, RouteNo FK, GarageID FK)

Route

Route(RouteNo PK, AvgPassengersPerDay)

Town

Town(TownID PK, TownName)

Route_Town

Route_Town(RouteNo FK, TownID FK)

PRIMARY KEY (RouteNo, TownID)

Driver

Driver(EmpNo PK, Name, Address, Phone)

Stage

Stage(StageID PK, RouteNo FK)

Driver_Stage

Driver_Stage(EmpNo FK, StageID FK)

PRIMARY KEY (EmpNo, StageID)

Garage

Garage(GarageID PK, Location, TownID FK)



👉 Story → Entity → Attribute → Relationship → ER Diagram → Table

5.2 How to identify the entity?

5.3 How to identify the attribute?

5.4 How to identify the relationship?

5.5 How to identify the cardinality?

1. How to identify an Entity?

An **entity** is a real-world object or concept that has independent existence.

How to identify:

- Look for **nouns** in the problem statement (person, place, thing, event).
- It must have **meaningful data to store**.
- It should be **distinguishable** from other objects.
- It usually becomes a **table** in a database.

Examples:

- Student, Bus, Route, Driver, Town

2. How to identify an Attribute?

An **attribute** is a property or characteristic of an entity.

How to identify:

- Ask: *What information describes the entity?*
- Attributes are usually **descriptive details** (fields).
- Often appear as **adjectives or data values**.

Examples:

- Student → Name, ID, Age
- Bus → Bus_ID, Capacity, Type
- Route → Route_ID, Start_Point, End_Point

3. How to identify a Relationship?

A **relationship** shows how two or more entities are connected.

How to identify:

- Look for **verbs or actions** in the problem statement.
- Ask: *How are entities interacting?*
- It connects entities logically.

Examples:

- Student **enrolls in** Course
- Bus **assigned to** Route
- Driver **drives** Bus

4. How to identify Cardinality?

Cardinality defines how many instances of one entity relate to another.

Common types:

- **One-to-One (1:1)** → One entity relates to one other entity
- **One-to-Many (1:N)** → One entity relates to many
- **Many-to-Many (M:N)** → Many entities relate to many

How to identify:

- Ask questions like:
 - Can one entity relate to many others?
 - Or only one?
- Analyze the business rules in the problem.

Examples:

- One Department → Many Employees (1:N)
- Many Students → Many Courses (M:N)

বাংলা (Bangla)

১. Entity কিভাবে চিহ্নিত করবেন?

Entity হলো বাস্তব জীবনের কোনো বস্তু বা ধারণা যার স্বাধীন অস্তিত্ব আছে।

চিহ্নিত করার উপায়:

- সমস্যার মধ্যে থাকা **noun (নামবাচক শব্দ)** খুঁজুন।
- যার সম্পর্কে ডাটা সংরক্ষণ করা প্রয়োজন।
- যা অন্য কিছু থেকে আলাদা করে চিহ্নিত করা যায়।
- সাধারণত এটি ডাটাবেসে **table** হয়।

উদাহরণ:

- Student, Bus, Route, Driver, Town

২. Attribute কিভাবে চিহ্নিত করবেন?

Attribute হলো কোনো entity-এর বৈশিষ্ট্য বা গুণ।

চিহ্নিত করার উপায়:

- প্রশ্ন করুন: *এই entity সম্পর্কে কী তথ্য দরকার?*
- এগুলো সাধারণত entity-কে describe করে।
- noun বা descriptive data হিসেবে থাকে।

উদাহরণ:

- Student → Name, ID, Age
- Bus → Bus_ID, Capacity
- Route → Route_ID, Start, End

৩. Relationship কিভাবে চিহ্নিত করবেন?

Relationship হলো entity গুলোর মধ্যে সম্পর্ক বা সংযোগ।

চিহ্নিত করার উপায়:

- সমস্যার মধ্যে থাকা **verb (ক্রিয়া)** খুঁজুন।
- প্রশ্ন করুন: *entity গুলো কীভাবে একে অপরের সাথে যুক্ত?*
- এটি entity গুলোর মধ্যে connection তৈরি করে।

উদাহরণ:

- Student **enrolls in** Course
- Bus **assigned to** Route
- Driver **drives** Bus

৪. Cardinality কিভাবে চিহ্নিত করবেন?

Cardinality বোঝায় এক entity আরেক entity-এর সাথে কতগুলো সম্পর্ক রাখে।

প্রকারভেদ:

- **One-to-One (1:1)**

- One-to-Many (1:N)
- Many-to-Many (M:N)

চিহ্নিত করার উপায়:

- প্রশ্ন করুন:
 - এক entity কি অনেকগুলোর সাথে সম্পর্ক রাখতে পারে?
 - নাকি একটির সাথে?
- business rule analysis করুন।

উদাহরণ:

- One Department → Many Employees (1:N)
- Many Students → Many Courses (M:N)

5.6 How to take the primary key?

How to choose (take) a Primary Key?

A **Primary Key (PK)** is an attribute (or a set of attributes) that uniquely identifies each record in a table.

Steps to identify a Primary Key:

1. **Check uniqueness**
 - The attribute must uniquely identify each entity instance.
 - No duplicate values allowed.
2. **Check NOT NULL**
 - Primary key values cannot be NULL.
3. **Choose minimal attributes**
 - Prefer a single attribute if possible.
 - Avoid unnecessary composite keys unless required.
4. **Ensure stability**
 - The value should not change frequently.
5. **Avoid descriptive attributes**
 - Avoid using attributes like Name, Address (they may repeat or change).
 - Prefer IDs like Student_ID, Bus_ID, etc.

Examples:

- Student → Student_ID (Primary Key)
- Bus → Bus_ID (Primary Key)
- Course → Course_ID (Primary Key)

Composite Primary Key (if needed):

- When a single attribute is not enough, combine multiple attributes.
- Example: Enrollment(Student_ID, Course_ID)

বাংলা (Bangla)

Primary Key কিভাবে নির্বাচন করবেন?

Primary Key হলো এমন একটি attribute যা প্রতিটি record কে uniquely identify করে।

Primary Key নেওয়ার নিয়ম:

1. **Uniqueness নিশ্চিত করতে হবে**
 - প্রতিটি value আলাদা হতে হবে।
 - duplicate থাকা যাবে না।
2. **NULL হবে না**

- Primary key এর value কখনো NULL হতে পারবে না।
- 3. **Minimal attribute ব্যবহার করুন**
 - সম্ভব হলে single attribute ব্যবহার করুন।
 - অপয়োজনীয় composite key এড়িয়ে চলুন।
- 4. **Stable হতে হবে**
 - value বারবার পরিবর্তন হওয়া উচিত না।
- 5. **Descriptive attribute এড়ান**
 - Name, Address এর মতো attribute সাধারণত repeat হতে পারে।
 - তাই ID ব্যবহার করা ভালো (Student_ID, Bus_ID ইত্যাদি)।

উদাহরণ:

- Student → Student_ID (Primary Key)
- Bus → Bus_ID (Primary Key)
- Course → Course_ID (Primary Key)

Composite Primary Key:

- যখন একটি attribute যথেষ্ট নয়, তখন একাধিক attribute একসাথে ব্যবহার করা হয়।
- উদাহরণ: Enrollment(Student_ID, Course_ID)

5.7 How to make relation and put in ER diagram?

How to make a Relationship and represent it in an ER Diagram?

A **relationship** shows how two or more entities are connected in a database.

Steps to create a relationship in an ER Diagram:

1. **Identify Entities**
 - First, find all entities (nouns) in the problem.
 - Example: Student, Course, Teacher
2. **Find the Relationship (Verb)**
 - Look for action words (verbs) connecting entities.
 - Example: Student *enrolls in* Course
3. **Draw Entities as Rectangles**
 - Each entity is represented by a rectangle.
4. **Draw Relationship as a Diamond**
 - The relationship is shown as a diamond shape between entities.
 - Label it with a verb (e.g., enrolls, assigns, drives).
5. **Connect Entities to Relationship**
 - Use lines to connect entities to the relationship diamond.
6. **Add Cardinality**
 - Show how many instances relate:
 - 1:1
 - 1:N
 - M:N
 - Write near the connecting lines.
7. **Add Attributes (optional)**
 - Attributes are shown as ovals connected to entities or relationships.

Example:

- Entities: Student, Course
- Relationship: Student *enrolls in* Course
- Cardinality: Many-to-Many (M:N)

ER Diagram structure:

- Student — (enrolls in) — Course

বাংলা (Bangla)

ER Diagram-এ Relationship কিভাবে তৈরি ও দেখাবেন?

Relationship হলো entity গুলোর মধ্যে সংযোগ।

ER Diagram-এ Relationship তৈরি করার ধাপ:

1. **Entity চিহ্নিত করুন**
 - সমস্যার মধ্যে থাকা noun গুলো খুঁজে বের করুন।
 - যেমন: Student, Course, Teacher
2. **Relationship (verb) খুঁজুন**
 - Entity গুলোর মধ্যে action/verb খুঁজুন।
 - যেমন: Student *enrolls in* Course
3. **Entity আঁকুন (Rectangle)**
 - প্রতিটি entity একটি rectangle আকারে আঁকা হয়।
4. **Relationship আঁকুন (Diamond)**
 - Entity গুলোর মাঝে diamond shape ব্যবহার করুন।
 - Relationship এর নাম verb আকারে লিখুন।
5. **Entity গুলোকে Relationship এর সাথে connect করুন**
 - Line দিয়ে যুক্ত করুন।
6. **Cardinality যোগ করুন**
 - সম্পর্ক কত ধরনের তা দেখান:
 - 1:1
 - 1:N
 - M:N
 - Line এর পাশে লিখুন।
7. **Attributes যোগ করুন (optional)**
 - Oval shape দিয়ে attributes দেখানো হয়।

উদাহরণ:

- Entity: Student, Course
- Relationship: Student *enrolls in* Course
- Cardinality: Many-to-Many (M:N)

ER structure:

- Student — (enrolls in) — Course

5.8 In a relationship how many tables we need including main data tables and can we reduce it.

How many tables are needed in a relationship?

The number of tables depends on the **type of relationship (cardinality)** between entities.

1. One-to-One (1:1)

- Usually **2 tables** (one for each entity).
- Sometimes can be merged into **1 table** if appropriate.

Example:

- Person ↔ Passport
- You can keep:
 - Person table
 - Passport table
- Or combine into one table if the relationship is tight.

2. One-to-Many (1:N)

- Usually **2 tables**.
- The **foreign key** is placed in the "many" side table.

Example:

- Department → Employee
- Tables:
 - Department (1)
 - Employee (N, contains Department_ID as FK)

3. Many-to-Many (M:N)

- Requires **3 tables**:
 - Table for Entity A
 - Table for Entity B
 - **Junction/Bridge table** (to manage the relationship)

Example:

- Student ↔ Course
- Tables:
 - Student
 - Course
 - Enrollment (Student_ID, Course_ID)

Can we reduce the number of tables?

Yes, but only in some cases:

- **1:1 relationship** → can sometimes be merged into a single table.
- **1:N relationship** → cannot meaningfully reduce tables without losing normalization.
- **M:N relationship** → cannot avoid the junction table in proper relational design.

Summary:

- 1:1 → 1 or 2 tables
- 1:N → 2 tables
- M:N → 3 tables (minimum)

বাংলা (Bangla)

Relationship-এ কতগুলো table লাগে?

Relationship-এর ধরন (cardinality) অনুযায়ী table এর সংখ্যা নির্ভর করে।

১. One-to-One (1:1)

- সাধারণত ২টি table লাগে (প্রতিটি entity এর জন্য একটি করে)।
- কিছু ক্ষেত্রে ১টি table-এ merge করা যায়।

উদাহরণ:

- Person ↔ Passport
- আলাদা table রাখা যায়:
 - Person table
 - Passport table
- অথবা একসাথে merge করা সম্ভব।

২. One-to-Many (1:N)

- সাধারণত ২টি table লাগে।
- "Many" side এ foreign key রাখা হয়।

উদাহরণ:

- Department → Employee
- Table:
 - Department (1)
 - Employee (N, এখানে Department_ID FK থাকে)

৩. Many-to-Many (M:N)

- সাধারণত ৩টি table লাগে:
 - Entity A table
 - Entity B table
 - Junction/Bridge table

উদাহরণ:

- Student ↔ Course
- Table:
 - Student
 - Course
 - Enrollment (Student_ID, Course_ID)

Table কমানো যায় কি?

- 1:1 relationship → কখনো কখনো ১টি table করা যায়
- 1:N relationship → সাধারণত কমানো যায় না
- M:N relationship → junction table ছাড়া করা যায় না

Summary:

- 1:1 → ১ বা ২টি table
- 1:N → ২টি table
- M:N → ৩টি table (minimum)

5.9 Difference between truncate and drop.

Difference between TRUNCATE and DROP

Feature	TRUNCATE	DROP
Purpose	Removes all records from a table	Deletes the entire table structure
Data	Data is removed	Data + structure both removed
Table structure	Remains	Removed completely
WHERE clause	Not allowed	Not applicable
Speed	Faster	Fast but more destructive
Rollback	Limited (depends on DBMS/transaction)	Cannot rollback in most cases
Identity/Auto increment	Reset (in many DBMS)	Table no longer exists
Use case	Clear all data but keep table	Remove table permanently
Example: - TRUNCATE table Student; → Deletes all rows, table still exists - DROP table Student; → Deletes the entire table from database		
বাংলা (Bangla) TRUNCATE এবং DROP এর মধ্যে পার্থক্য		
বিষয়	TRUNCATE	DROP
উদ্দেশ্য	টেবিলের সব data মুছে ফেলা	টেবিল সম্পূর্ণ মুছে ফেলা
Data	শুধু data delete হয়	data + structure উভয়ই delete হয়
Table structure	থাকে	সম্পূর্ণ remove হয়
WHERE clause	ব্যবহার করা যায় না	প্রযোজ্য নয়
Speed	দ্রুত	দ্রুত কিন্তু বেশি destructive
Rollback	সীমিত (DBMS নির্ভর)	সাধারণত rollback করা যায় না
Auto increment	অনেক ক্ষেত্রে reset হয়	টেবিল আর থাকে না
ব্যবহার	শুধু data পরিষ্কার করতে	টেবিল পুরো delete করতে
উদাহরণ: - TRUNCATE table Student; → সব row মুছে যাবে, table থাকবে - DROP table Student; → table পুরো database থেকে মুছে যাবে		

5.10 What is subquery and its practice example.

What is a Subquery? A subquery (also called a nested query) is a query written inside another SQL query . - The inner query is executed first. - Its result is used by the outer query. - It is usually placed inside WHERE, FROM, or SELECT.
Types of Subqueries: Single-row subquery → returns one value Multiple-row subquery → returns multiple values Correlated subquery → depends on the outer query
Practice Examples

Example 1: Using subquery in WHERE

Find students who scored above the average mark.

```
SELECT Name, Marks
FROM Student
WHERE Marks > (SELECT AVG(Marks) FROM Student);
```

Example 2: Find employees in a department

Find employees who belong to the "IT" department.

```
SELECT Name
FROM Employee
WHERE Dept_ID = (
    SELECT Dept_ID
    FROM Department
    WHERE Dept_Name = 'IT'
);
```

Example 3: Subquery with IN

Find students who are enrolled in any course.

```
SELECT Name
FROM Student
WHERE Student_ID IN (
    SELECT Student_ID
    FROM Enrollment
);
```

বাংলা (Bangla)

Subquery কী?

Subquery হলো একটি query যা আরেকটি query-এর ভিতরে লেখা থাকে।

- ভিতরের query আগে execute হয়
- তার result বাইরের query ব্যবহার করে
- সাধারণত WHERE, FROM, বা SELECT এর মধ্যে থাকে

Subquery এর ধরন:

- **Single-row subquery** → একটাই value return করে
- **Multiple-row subquery** → একাধিক value return করে
- **Correlated subquery** → outer query এর উপর নির্ভর করে

Practice Examples

উদাহরণ ১: WHERE এ subquery

Average marks এর চেয়ে বেশি marks পাওয়া student খুঁজুন।

```
SELECT Name, Marks
FROM Student
WHERE Marks > (SELECT AVG(Marks) FROM Student);
```

উদাহরণ ২: Department অনুযায়ী employee

IT department এর employee খুঁজুন।

```
SELECT Name
```

```
FROM Employee
WHERE Dept_ID = (
  SELECT Dept_ID
  FROM Department
  WHERE Dept_Name = 'IT'
);
```

উদাহরণ ৩: IN দিয়ে subquery

যেসব student কোনো course এ enrolled আছে।

```
SELECT Name
FROM Student
WHERE Student_ID IN (
  SELECT Student_ID
  FROM Enrollment
);
```

5.11 What is arithmetic function?

What is an Arithmetic Function?

An **arithmetic function** is a function used in SQL (or general computing) to perform **mathematical calculations** on numeric data. These functions help manipulate numbers such as addition, subtraction, multiplication, division, rounding, etc.

Common Arithmetic Operations (Operators)

These are basic arithmetic operations used in SQL:

- + → Addition
- - → Subtraction
- * → Multiplication
- / → Division
- % → Modulus (remainder)

Examples of Arithmetic Functions/Operations in SQL

1. Addition

```
SELECT Salary + Bonus AS Total_Income
FROM Employee;
```

2. Subtraction

```
SELECT Salary - Tax AS Net_Salary
FROM Employee;
```

3. Multiplication

```
SELECT Price * Quantity AS Total_Cost
FROM Orders;
```

4. Division

```
SELECT Total_Amount / Items AS Average_Price
FROM Orders;
```

5. Modulus (Remainder)

```
SELECT 10 % 3 AS Remainder;
```

Built-in Mathematical Functions (Advanced)

Some DBMS also provide built-in arithmetic/math functions:

- $ABS(x) \rightarrow$ Absolute value
- $ROUND(x, n) \rightarrow$ Round a number to n decimal places
- $CEIL(x) \rightarrow$ Round up
- $FLOOR(x) \rightarrow$ Round down
- $POWER(x, y) \rightarrow$ x raised to power y
- $SQRT(x) \rightarrow$ Square root

Example:

```
SELECT ROUND(Salary, 2)
FROM Employee;
```

Key Points:

- Arithmetic functions work only on **numeric data**.
- They are commonly used in **SELECT queries**.
- Can be used with **columns, constants, or expressions**.
- Very useful for **calculations, reports, and analysis**.

বাংলা (Bangla)

Arithmetic Function কী?

Arithmetic function হলো এমন function যা numeric data এর উপর **গাণিতিক হিসাব** করার জন্য ব্যবহার করা হয়।

এগুলো দিয়ে যোগ, বিয়োগ, গুণ, ভাগ, rounding ইত্যাদি করা যায়।

Arithmetic Operators (মৌলিক অপারেশন)

SQL এ সাধারণ arithmetic operators:

- $+$ $\rightarrow$ যোগ (Addition)
- $-$ $\rightarrow$ বিয়োগ (Subtraction)
- $*$ $\rightarrow$ গুণ (Multiplication)
- $/$ $\rightarrow$ ভাগ (Division)
- $\%$ $\rightarrow$ ভাগশেষ (Modulus)

SQL এ Examples

১. যোগ (Addition)

```
SELECT Salary + Bonus AS Total_Income
FROM Employee;
```

২. বিয়োগ (Subtraction)

```
SELECT Salary - Tax AS Net_Salary
FROM Employee;
```

৩. গুণ (Multiplication)

```
SELECT Price * Quantity AS Total_Cost
FROM Orders;
```

৪. ভাগ (Division)

```
SELECT Total_Amount / Items AS Average_Price
FROM Orders;
```

৫. ভাগশেষ (Modulus)

```
SELECT 10 % 3 AS Remainder;
```

Built-in Mathematical Functions

কিছু built-in function আছে:

- $ABS(x)$ → absolute value
- $ROUND(x, n)$ → নির্দিষ্ট decimal পর্যন্ত round
- $CEIL(x)$ → উপরের পূর্ণসংখ্যা
- $FLOOR(x)$ → নিচের পূর্ণসংখ্যা
- $POWER(x, y)$ → power
- $SQRT(x)$ → square root

উদাহরণ:

```
SELECT ROUND(Salary, 2)
FROM Employee;
```

গুরুত্বপূর্ণ বিষয়:

- Arithmetic function শুধু **numeric data** এর উপর কাজ করে
- সাধারণত **SELECT query** তে ব্যবহার করা হয়
- calculation, report তৈরিতে খুবই গুরুত্বপূর্ণ

5.12 What is union and other join operation?

1. What is UNION?

UNION is a SQL operation used to **combine the result sets of two or more SELECT queries** into a single result.

Key Rules of UNION:

- Each SELECT must have the **same number of columns**.
- Corresponding columns must have **compatible data types**.
- Removes **duplicate rows** by default.
- Use **UNION ALL** to include duplicates.

Example:

```
SELECT Name FROM Student
UNION
```

```
SELECT Name FROM Teacher;
```

👉 Combines names from both tables and removes duplicates.

```
SELECT Name FROM Student
UNION ALL
```

```
SELECT Name FROM Teacher;
```

👉 Keeps duplicate values also.

2. What are JOIN operations?

JOIN is used to **combine rows from two or more tables based on a related column** (usually a key).

Types of JOINS

a) INNER JOIN

- Returns only matching records from both tables.

```
SELECT Student.Name, Course.Course_Name
FROM Student
```


INNER JOIN Enrollment

ON Student.Student_ID = Enrollment.Student_ID

INNER JOIN Course

ON Enrollment.Course_ID = Course.Course_ID;

👉 Only students who are enrolled in courses will appear.

b) LEFT JOIN (LEFT OUTER JOIN)

- Returns all records from the left table and matched records from the right table.
- Unmatched right side = NULL.

SELECT Student.Name, Course.Course_Name

FROM Student

LEFT JOIN Enrollment

ON Student.Student_ID = Enrollment.Student_ID

LEFT JOIN Course

ON Enrollment.Course_ID = Course.Course_ID;

👉 All students will appear, even if not enrolled.

c) RIGHT JOIN (RIGHT OUTER JOIN)

- Returns all records from the right table and matching from the left.

SELECT Student.Name, Course.Course_Name

FROM Student

RIGHT JOIN Enrollment

ON Student.Student_ID = Enrollment.Student_ID;

d) FULL JOIN (FULL OUTER JOIN)

- Returns all records from both tables.
- Matches where possible, otherwise NULL.

SELECT *

FROM Student

FULL JOIN Enrollment

ON Student.Student_ID = Enrollment.Student_ID;

UNION vs JOIN (Difference)

Feature	UNION	JOIN
Purpose	Combines result sets	Combines tables based on relation
Data	Rows are combined vertically	Columns are combined horizontally
Requirement	Same columns	Related columns (keys)
Duplicate handling	Removes duplicates (UNION)	Does not remove duplicates

বাংলা (Bangla)

১. UNION কী?

UNION হলো এমন একটি SQL operation যা একাধিক SELECT query এর result একত্র করে একটি result set তৈরি করে।

UNION এর নিয়ম:

- প্রতিটি SELECT এ **same number of column** থাকতে হবে
- column গুলোর data type compatible হতে হবে

- defaultভাবে duplicate remove করে
- duplicate রাখতে চাইলে **UNION ALL** ব্যবহার করতে হয়

উদাহরণ:

```
SELECT Name FROM Student
UNION
```

```
SELECT Name FROM Teacher;
```

👉 Student ও Teacher এর নাম একসাথে দেখাবে, duplicate বাদ যাবে।

```
SELECT Name FROM Student
UNION ALL
```

```
SELECT Name FROM Teacher;
```

👉 duplicate সহ সব দেখাবে।

২. JOIN কী?

JOIN ব্যবহার করা হয় একাধিক table কে **common column (key)** এর মাধ্যমে যুক্ত করতে।

JOIN এর ধরন

a) INNER JOIN

- শুধু matching record দেখায়।

```
SELECT Student.Name, Course.Course_Name
FROM Student
INNER JOIN Enrollment
ON Student.Student_ID = Enrollment.Student_ID
INNER JOIN Course
ON Enrollment.Course_ID = Course.Course_ID;
```

👉 শুধু enrolled students দেখাবে।

b) LEFT JOIN

- Left table এর সব data দেখায়, right table match না করলে NULL দেয়।

```
SELECT Student.Name, Course.Course_Name
FROM Student
LEFT JOIN Enrollment
ON Student.Student_ID = Enrollment.Student_ID;
```

👉 সব student দেখাবে, enrolled না হলেও।

c) RIGHT JOIN

- Right table এর সব data দেখায়।

d) FULL JOIN

- দুই table এর সব data দেখায় (match থাকলে match, না থাকলে NULL)।

UNION vs JOIN পার্থক্য

বিষয়	UNION	JOIN
উদ্দেশ্য	result set combine করা	table combine করা
কাজের ধরন	vertical merge	horizontal merge
প্রয়োজন	same columns	related keys

duplicate remove করে (UNION) remove করে না

5.13 Join vs union operation (for 3*3 tow table).

Join vs Union (Using Two 3×3 Tables)

Assume we have two tables:

Table A (3×3)

ID Name Dept

1	A	IT
2	B	HR
3	C	IT

Table B (3×3)

ID Name Dept

4	D	FIN
5	E	IT
6	F	HR

◆ 1. UNION Operation

Concept:

- Combines **rows** from both tables.
- Works **vertically** (stacking rows).
- Requires:
 - Same number of columns
 - Compatible data types
- Removes duplicates (by default).

Result of UNION:

```
SELECT ID, Name, Dept FROM TableA  
UNION
```

```
SELECT ID, Name, Dept FROM TableB;
```

Output (combined rows):

ID Name Dept

1	A	IT
2	B	HR
3	C	IT
4	D	FIN
5	E	IT
6	F	HR

👉 Rows are simply stacked together.

◆ 2. JOIN Operation

Concept:

- Combines **columns** from two tables.
- Works **horizontally** using a related column (key).
- Requires a **common attribute** (e.g., ID).

Example: INNER JOIN

Assume both tables share a common column (ID or Dept logic).

```
SELECT A.ID, A.Name, B.Dept
```

```
FROM TableA A
```

```
INNER JOIN TableB B
```

```
ON A.Dept = B.Dept;
```

Output (matching Dept):

ID Name Dept

1 A IT

3 C IT

2 B HR

👉 Only matching records are shown.

🔥 Key Differences (Join vs Union)

Feature	UNION	JOIN
Operation type	Vertical combination	Horizontal combination
Combines	Rows	Columns
Requirement	Same number of columns	Common key/relationship
Result size	Rows increase	Columns increase
Duplicate handling	Removes duplicates (UNION)	No automatic removal
Use case	Merge similar datasets	Relate different tables

🧠 Simple Intuition

- **UNION** → “Stack tables” (row-wise merge)
- **JOIN** → “Connect tables” (column-wise merge using keys)

বাংলা (Bangla)

Join vs Union (Two 3×3 Table দিয়ে)

ধরি আমাদের দুইটি table আছে:

Table A (3×3)

ID Name Dept

1 A IT

2 B HR

3 C IT

Table B (3×3)

ID Name Dept

4 D FIN

5 E IT

6 F HR

♦ ১. UNION Operation

ধারণা:

- দুই table এর rows একসাথে যোগ করে

- কাজ করে **verticalভাবে**
- শর্ত:
 - একই number of columns থাকতে হবে
 - data type compatible হতে হবে
- duplicate automatically remove করে

Example:

```
SELECT ID, Name, Dept FROM TableA
UNION
SELECT ID, Name, Dept FROM TableB;
```

Result:

ID Name Dept

```
1 A      IT
2 B      HR
3 C      IT
4 D      FIN
5 E      IT
6 F      HR
```

👉 দুই table এর row গুলো একসাথে বসে যায়।

♦ **২. JOIN Operation**

ধারণা:

- দুই table কে **common column (key)** দিয়ে যুক্ত করে
- কাজ করে **horizontalভাবে**
- columns combine করে

Example: INNER JOIN

```
SELECT A.ID, A.Name, B.Dept
FROM TableA A
INNER JOIN TableB B
ON A.Dept = B.Dept;
```

Result:

ID Name Dept

```
1 A      IT
3 C      IT
2 B      HR
```

👉 শুধু matching data দেখায়।

🔥 **পার্থক্য (Summary)**

বিষয়	UNION	JOIN
কাজ	row যোগ করা	column যোগ করা
ধরন	vertical merge	horizontal merge
প্রয়োজন	same columns	common key
result	rows বাড়ে	columns বাড়ে
duplicate remove করে		করে না

5.14 Join vs set operation.

Join vs Set Operations in DBMS

Both **JOIN** and **SET operations** are used to combine data from tables, but they work in different ways.

◆ 1. JOIN Operations

Concept:

JOIN is used to **combine rows from two or more tables based on a related column (key)**.

How it works:

- Combines data **horizontally (columns)**
- Requires a **common attribute (primary key / foreign key)**
- Used when tables are related

Example:

```
SELECT Student.Name, Course.Course_Name
FROM Student
INNER JOIN Enrollment
ON Student.Student_ID = Enrollment.Student_ID
INNER JOIN Course
ON Enrollment.Course_ID = Course.Course_ID;
```

👉 Output combines columns from multiple tables based on relationships.

◆ 2. Set Operations

Concept:

Set operations are used to **combine results of two SELECT queries**.

Common Set Operations:

- **UNION**
- **UNION ALL**
- **INTERSECT**
- **EXCEPT (MINUS)**

How it works:

- Combines data **vertically (rows)**
- No need for relationship between tables
- Requires same number of columns and compatible data types

Example of UNION:

```
SELECT Name FROM Student
UNION
SELECT Name FROM Teacher;
```

👉 Combines rows from both queries.

Example of INTERSECT:

```
SELECT Name FROM Student
INTERSECT
SELECT Name FROM Teacher;
```

👉 Returns common records in both queries.

Example of EXCEPT:

SELECT Name FROM Student

EXCEPT

SELECT Name FROM Teacher;

👉 Returns records in Student but not in Teacher.

🔥 Key Differences

Feature	JOIN	SET Operations
Purpose	Combine related tables	Combine query results
Type of combination	Horizontal (columns)	Vertical (rows)
Relationship required	Yes (keys)	No relationship required
Input	Tables	SELECT queries
Result	Wide table (more columns)	Tall result (more rows)
Operations included	INNER, LEFT, RIGHT, FULL JOIN UNION, INTERSECT, EXCEPT	

🧠 Simple Understanding

- **JOIN** → connects tables using relationships (columns expand)
- **SET** → stacks or compares results of queries (rows expand)

বাংলা (Bangla)

Join vs Set Operation (DBMS)

JOIN এবং **SET operation** দুইটাই table data combine করার জন্য ব্যবহৃত হয়, কিন্তু কাজের ধরন আলাদা।

♦ ১. JOIN Operation

ধারণা:

JOIN ব্যবহার করা হয় একাধিক table কে **common key** দিয়ে যুক্ত করতে।

কাজের ধরন:

- **horizontalভাবে (columns)** data combine করে
- relationship (primary key / foreign key) লাগে
- related tables এর জন্য ব্যবহার হয়

উদাহরণ:

SELECT Student.Name, Course.Course_Name

FROM Student

INNER JOIN Enrollment

ON Student.Student_ID = Enrollment.Student_ID;

👉 columns combine হয় relation অনুযায়ী।

♦ ২. SET Operation

ধারণা:

SET operation ব্যবহার করা হয় একাধিক **SELECT query** এর result combine করতে।

প্রধান SET operations:

- UNION
- UNION ALL
- INTERSECT

- EXCEPT

কাজের ধরন:

- **verticalভাবে (rows)** data combine করে
- table এর মধ্যে relationship দরকার হয় না
- same number of column থাকতে হয়

উদাহরণ (UNION):

SELECT Name FROM Student

UNION

SELECT Name FROM Teacher;

👉 দুই query এর row একসাথে দেখায়।

INTERSECT:

👉 common data বের করে

SELECT Name FROM Student

INTERSECT

SELECT Name FROM Teacher;

EXCEPT:

👉 এক table এ আছে কিন্তু অন্যটিতে নেই এমন data

SELECT Name FROM Student

EXCEPT

SELECT Name FROM Teacher;

🔥 পার্থক্য

বিষয়	JOIN	SET Operation
উদ্দেশ্য	related table combine	query result combine
কাজ	column যোগ	row যোগ
relation লাগে লাগে		লাগে না
input	table	SELECT query
result	wide (columns বাড়়ে)	tall (rows বাড়়ে)
types	INNER, LEFT, RIGHT, FULL UNION, INTERSECT, EXCEPT	

🧠 সহজভাবে

- JOIN → table connect করে (column বাড়়ে)
- SET → result stack করে (row বাড়়ে)

5.15 What is